

# Performance and Communication Cost of Hardware Accelerators for Hashing in Post-Quantum Cryptography

PATRICK KARL, TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany
JONAS SCHUPP, TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany
GEORG SIGL, TUM School of Computation, Information and Technology, Technical University Munich, Munich, Germany and Fraunhofer Institute for Applied and Integrated Security, Garching, Germany

SPHINCS+ is a signature scheme included in the first NIST post-quantum standard that bases its security on the underlying hash primitive. As most of the runtime of SPHINCS+ is caused by the evaluation of several hash- and pseudo-random functions, offloading this computation to dedicated hardware accelerators is a natural step. In this work, we evaluate different architectures for hardware acceleration of such a hash primitive with respect to its use-case and evaluate them in the context of SPHINCS+. We attach hardware accelerators for different hash primitives (SHAKE256 and Ascon-Xof for both full and round-reduced versions) to CPU interfaces having different transfer speeds. We show that for most use-cases, data transfer determines the overall performance if accelerators are equipped with FIFOs and that reducing the number of rounds in the permutation does not necessarily lead to significant performance improvements when using hardware acceleration.

This work extends on a conference paper accepted at COSADE'24, first published in [19], and written by the same authors, where different architectures for hardware accelerators of hash functions are benchmarked and evaluated for SPHINCS+ as a case study. In this article, we provide results for additional parameter sets for SPHINCS+ and improve the performance of one of the accelerators by adding an additional RISC-V instruction for faster absorption. We then extend the performance benchmark by including the algorithms CRYSTALS-Kyber, CRYSTALS-Dilithium, and Falcon. Finally, we provide a power/energy comparison for the accelerators.

CCS Concepts: • **Security and privacy → Hardware security implementation**; *Hash functions and message authentication codes;*

Additional Key Words and Phrases: NIST PQC, post-quantum cryptography, RISC-V HW/SW co-design, hardware acceleration

The authors acknowledge the financial support by the Federal Ministry of Education and Research of Germany in the program of "Souverän. Digital. Vernetzt.". Joint project 6G-life, project identification number: 16KIS002.
Authors' Contact Information: Patrick Karl, TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany; e-mail: patrick.karl@tum.de; Jonas Schupp, TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany; e-mail: jonas.schupp@tum.de; Georg Sigl, TUM School of Computation, Information and Technology, Technical University Munich, Munich, Germany and Fraunhofer Institute for Applied and Integrated Security, Garching, Germany; e-mail: sigl@tum.de.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2025 Copyright held by the owner/author(s).
ACM 1539-9087/2025/09-ART66
https://doi.org/10.1145/3676965

**ACM Reference Format:**
Patrick Karl, Jonas Schupp, and Georg Sigl. 2025. Performance and Communication Cost of Hardware Accelerators for Hashing in Post-Quantum Cryptography. *ACM Trans. Embedd. Comput. Syst.* 24, 5, Article 66 (September 2025), 31 pages. https://doi.org/10.1145/3676965

## 1 Introduction

The umbrella term **Post-Quantum Cryptography (PQC)** describes cryptographic algorithms that provide security in the presence of large-scale quantum computers. In 2016, the **National Institute of Standards and Technology (NIST)** started a public standardization process to analyze and evaluate quantum-secure algorithms for **Key-Encapsulation Mechanisms (KEMs)**, as well as **Digital Signature Algorithms (DSAs)**. Recently, NIST selected a first set of algorithms for standardization [25]. One scheme, i.e., CRYSTALS-Kyber [8], is from the category of KEMs, and three of them are DSAs, namely CRYSTALS-Dilithium [10], Falcon [11], and SPHINCS+ [5]. In order to further diversify its portfolio and to find schemes with smaller signatures, NIST opened another process for additional signature schemes with its first round of candidates announced in June 2023 [24].

Many PQC algorithms spend a substantial amount of computation time to hash data to a fix-sized digest or to generate pseudo-randomness to sample elements from certain distributions. From the set of NIST-selected algorithms, SPHINCS+ is from the class of hash-based signatures. Its security solely relies on the security of the instantiated hash function, and the choice for the hash function is flexible. One of the proposed instances uses the SHA-3 **Extendable Output Function (XOF)** SHAKE256. For this choice, the algorithm spends more than 95 % of its runtime inside the hash function when running on a commonly used microcontroller [17]. Moreover, using the SHA-3 functions for hashing and generation of pseudo-randomness is a usual choice also for other PQC algorithms. For instance, according to recent benchmarks on the ARM Cortex-M4, some of the new on-ramp signatures in the NIST competition spend more than half of their execution time within a symmetric primitive [16]. Thus, a generic hardware accelerator for the underlying hash primitive is a natural step to improve the performance and energy consumption of PQC schemes, and in particular hash-based schemes like SPHINCS+.

*Related Work.* There have been different HW/SW co-design approaches to accelerate the SHA-3 functions. In [13], a tightly coupled solution has been presented and evaluated (among others) for Kyber. This approach has been adapted in [18] to also accelerate Dilithium and parts of the Falcon signature scheme. In addition to that, dedicated SHA-3 co-processors have been employed to accelerate PQC algorithms, as shown in, e.g., [1, 4, 12, 31] (non-exhaustive).

Previous works investigated hardware implementations for SPHINCS+ but mostly focused on co-processors that compute the entire signature. For instance, the work in [2] presents a standalone, high-throughput co-processor using SHAKE256. In [6], another co-processor with area efficiency as the primary design goal is presented using SHA-2 as hash primitive. These standalone designs yield high performance but lack flexibility—they speed up a single algorithm, but their re-use for other schemes is limited. Hardware/software co-designs provide a tradeoff between performance and flexibility by offloading only computationally intensive operations to dedicated hardware but run the main algorithm in software. The work in [30] investigates the use of SPHINCS+ in the context of secure boot within a hardware–software co-design and extends the platform's SHA-2 core to improve the performance for hash-based signatures in general.

To the best of our knowledge, there is no systematic study that compares different architectures for the acceleration of hash algorithms in an HW/SW co-design. In addition to that, no

HW/SW co-design evaluating SPHINCS+ using the SHA-3 standard has been presented prior to our work in [19]. We note, however, that concurrent to this extended work, Saarinen published a co-processor for SPHINCS+ (as to be standardized as FIPS 205 SLH-DSA [23]) in [27] that accelerates not only the basic hash functionality but also the SPHINCS+ specific iterative hashing as well as input preparation. Consequently, it obtained better performance and overall efficiency.

*Contribution.* Based on our work in [19], we explore different design architectures for hardware accelerators and evaluate the advantages and disadvantages of these options with respect to resource cost and performance. For that, we use a publicly available 32 -bit micro-controller platform based on the open-source RISC-V **Instruction Set Architecture (ISA)** and include dedicated hardware extensions. We show that the choice for the most performant design option is application dependent. We use SPHINCS+ as a first case study and provide performance metrics for the resulting HW/SW co-design. Furthermore, we replace the SHAKE256 primitive with other alternatives like TurboSHAKE256 [7], a round-reduced SHAKE256 version, and functions from the Ascon [9] suite, the winner of the NIST **Lightweight Cryptography (LWC)** competition.

These contributions have been first published in the COSADE 2024 proceedings (LNCS, volume 14595, pp 221–239) by Springer Nature [19]:

— Design-space exploration and benchmark of hash-accelerator architectures on a 32 -bit RISC-V platform, considering the corresponding requirements of data transfer
— A practical analysis of the hidden (performance) cost of hash accelerators on a 32 -bit platform and the implications for SPHINCS+
— Performance evaluation for SPHINCS+ using hardware acceleration for different primitives, i.e., SHAKE256, TurboSHAKE256, Ascon-Xof, and Ascon-Xofa

In this extended work, we provide results for additional SPHINCS+ parameter sets and extend our design-space analysis of [19] to the PQC schemes Dilithium, Falcon, and Kyber. For performance improvements, we add a dedicated RISC-V instruction to increase the efficiency of the absorption phase of the hash function for the tightly coupled strategy presented in [13]. Finally, we provide a power and energy comparison of the accelerator architectures using a 22 nm ASIC technology. In summary, this extended work adds:

— Additional performance improvements using a dedicated RISC-V ISA extension
— Additional performance evaluation for Dilithium, Falcon, and Kyber using hardware acceleration for SHAKE256 and TurboSHAKE256
— Comparison of energy and power consumption for different accelerator architectures using Globalfoundries 22 nm FDSOI technology

*Organization.* Section 2 provides a brief overview of the SPHINCS+ signature scheme, the SHA-3 standard, and our evaluation setup. In Section 3 we explain different architectures for the investigated hardware accelerators and their parameterization and cost. A performance analysis of these architectures and their application to SPHINCS+ is conducted in Section 4. Section 5 investigates the usage of cryptographic primitives for SPHINCS+ that are not part of the NIST submission and discuss their implications on performance and further optimization strategies. While these sections are included in our work in [19], we provide Section 4.2 and Section 4.4 as additions to this extended work. In Section 6 we widen the scope of the analysis and include all to-be-standardized PQC algorithms (as of July 2024). We conduct a power and energy analysis for an ASIC target in Section 7. Both Section 6 and Section 7 provide new results exclusively to this work. Finally, Section 8 concludes this work.

## 2 Preliminaries

### 2.1 The SPHINCS+ Framework

SPHINCS+ [5] is a stateless hash-based PQC signature scheme that has been selected by NIST as a winner of their first PQC standardization effort [25]. It uses different families of hash and pseudo-random functions and solely relies on their (second) preimage resistance. The core idea of SPHINCS+ is to combine several layers of Merkle trees with **Winternitz One-Time Signature (WOTS+)** [15] key pairs on its leafs. In the last layer of the resulting so-called hypertree, a **Few-Time Signature (FTS)** scheme called **Forest Of Random Subsets (FORS)** [5] is employed. In a simplified view, the signature of SPHINCS+ consists of the FORS signature and a series of WOTS+ signatures on the selected path through the hypertree. A verifier can then verify the signatures on the selected path and re-compute and verify the root node of the hypertree. For details, we refer to the original work in [5].

In the construction of the trees within SPHINCS+, the following pseudo-random and message-digest functions are used, where $\mathbb{B}$ denotes the set of bytes:

$$\begin{aligned} \boldsymbol{PRF} &: \mathbb{B}^n \times \mathbb{B}^{32} \rightarrow \mathbb{B}^n \\ \boldsymbol{PRF_{msg}} &: \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^n \\ \boldsymbol{H_{msg}} &: \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^m. \end{aligned}$$

Furthermore, a hash function $\boldsymbol{T_l}$ with its two special cases $\boldsymbol{F} = \boldsymbol{T_1}$ and $\boldsymbol{H} = \boldsymbol{T_2}$ is defined as follows:

$$\boldsymbol{T_l} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{ln} \rightarrow \mathbb{B}^n.$$

The parameter $n \in \{16, 24, 32\}$ defines the output length (in bytes) of all the functions being used except for $H_{msg}$, which outputs an $m$-byte string. Likewise, the size of $n$ determines the NIST security levels I, III, and V. In this work, we consider parameter sets using the XOF SHAKE256 from the SHA-3 standard [26] as the underlying primitive, as specified in the SPHINCS+ NIST submission.[1] Therefore, we briefly explain its concept in Section 2.2.

### 2.2 Keccak and the SHA-3 Standard

SHAKE256 is based on the Keccak primitive and consists of a 1,600 -bit state on which the permutation function `keccak-f1600` is applied for 24 rounds. For all the functions defined in the SHA-3 standard [26], Keccak is used in a sponge construction. The 1,600 -bit state is divided into a rate $r$ and a capacity $c$ and is initialized with zeroes. The input data is split into several blocks $b_i$ of size $r$ that are consecutively absorbed into the state. Between the absorption of two blocks, the permutation function is applied. After absorbing all blocks, the output blocks $b_o$ (of size $r$) are squeezed, and if more than one block is requested, the state is permuted again after each output block.

SHA-3 defines several hash functions with fixed output lengths and two XOF functions with outputs of arbitrary length, i.e., SHAKE128 and SHAKE256.

### 2.3 Evaluation Platform

As a baseline platform for the evaluation in this work, we use the PULPino microcontroller platform[2] that instantiates the cv32e40p, a four-stage pipelined 32 -bit RISC-V core supporting the *RV32IMC* and optional *F* instruction set [14]. The open-source character of RISC-V and the PULPino microcontroller enables integrating arbitrary hardware extensions and accelerators. We evaluate our experiments on a Zynq UltraScale+ FPGA (xczu9eg-ffvb1156-2) using Vivado 2020.2

[1] https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions

[2] https://github.com/pulp-platform/pulpino

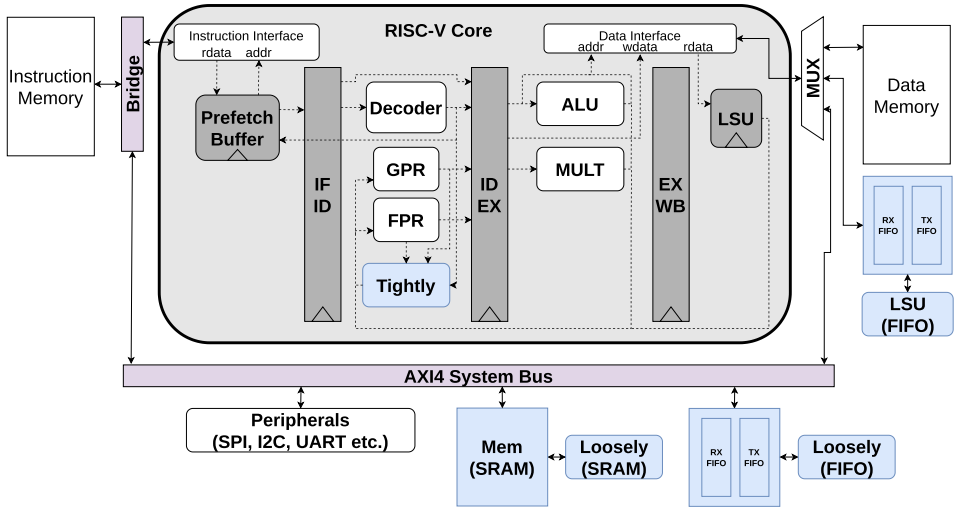



Fig. 1. Extended PULPino with accelerator architectures (blue).

as a synthesis tool. On this platform, the PULPino baseline implementation consumes 16,038 **Look-Up Tables (LUTs)** and 10,050 **Flip-Flops (FFs)** after synthesis and runs at a frequency of 150 MHz. It is noteworthy that none of the hardware extensions investigated in this work lead to a decrease in frequency and, thus, the reduction in cycle counts directly translates to performance gains in terms of latency. Software is compiled with the PULP compiler[3] with `-O3`.

## 3 Accelerator Architectures

Figure 1 shows the schematic of the PULPino microcontroller and different accelerator options. The accelerator architectures that are investigated in this work are colored blue. In general, a designer has different options to integrate custom logic, which mainly differ in the interfacing logic. For instance, one can integrate custom functionality *tightly* into the processor and extend the ISA with corresponding instructions. One can also connect a co-processor *loosely* to the system bus. Finally, the last option is to connect the co-processor to the **Load-Store Unit (LSU)** before bridging onto the system bus. For evaluation, we implemented the investigated hash cores in SystemVerilog and integrated them in the different architectures as described in the following sections. The hash cores themselves have a 32 -bit interface, automatically perform the padding in hardware, and compute one round of the corresponding permutation per clock cycle. The correct functionality of the cores has been verified both in simulation and on the FPGA by comparing the computed outputs against software reference implementations.

### 3.1 Tightly Coupled into Register Set

In [13], the SHA-3 round function `keccak-f1600` has been integrated tightly coupled into the register set of a RISC-V microcontroller. A fixed selection of registers that stores the entire Keccak state is chosen at design time and the logic of the round function is implemented in hardware—hardwired to the specific selection of registers. This concept is depicted in Figure 2. The advantage of this strategy is that it prevents load-store overheads that are required for loosely coupled solutions connected to a system bus. It also provides a flexible solution that can be reused for any function relying on the Keccak permutation, because only the round function itself

[3] https://github.com/pulp-platform/pulp-riscv-gnu-toolchain

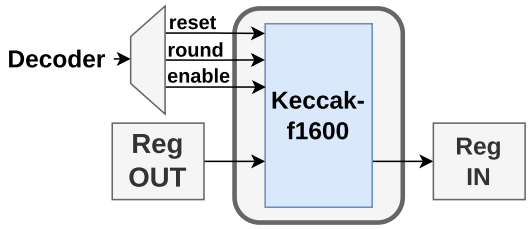



Fig. 2. Tightly coupled architecture as proposed in [13].

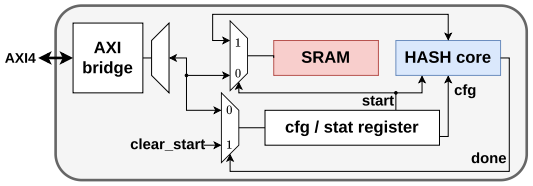



Fig. 3. Loosely coupled architecture using SRAM.

is implemented in hardware. Consequently, different functions (e.g., SHA-3 hash functions SHA3-256, SHA3-384, SHA3-512, SHAKE128, or SHAKE256) are run in software and only their costly permutation routine is replaced by a single custom RISC-V instruction per round. A drawback, however, is that for 32 -bit platforms, it requires the presence of both the **General-Purpose Registers (GPRs)** and the **Floating-Point Registers (FPRs)** to store the entire 1,600 -bit Keccak state. Besides that, the compilation toolchain must be adapted accordingly to support the custom instruction. Finally, the desired functions must be written in RISC-V assembly to prevent any corruption of registers during the execution of SHA-3 functions.

### 3.2 Loosely Coupled with SRAM

The simplest way of integrating a hardware co-processor is attaching it to the platform's system bus as shown in Figure 3. In this architecture, the address space is divided into a data section that enables writing into and reading from a dedicated SRAM, and a configuration/status register. For a hash co-processor, the procedure using this architecture is as follows: The user writes the input data to the SRAM via the system bus and configures the hash core. This includes, for instance, the desired mode of operation (e.g., SHA-3-256, SHAKE256, etc.) or the input and output length of the data. Finally, a signal *start* triggers the core to process the input data and the produced hash output is written into the SRAM. Afterward, the produced data can be read again via the AXI4 interface. The advantage of this approach is the easy integration into the platform and its accessibility in software—software drivers can be written in generic C, for instance. The drawback is that the core starts the computation after the data is fully written into the memory and the produced digest is read after the computation is finished. As we show in Section 4, this has a severe impact on performance for large data sizes. Nevertheless, it is notable that the additional SRAM memory not only is usable by the accelerator but also can be used by the RISC-V core when the accelerator is idle.

### 3.3 Loosely Coupled with FIFOs

Using **First-In-First-Outs (FIFOs)** as depicted in Figure 4 instead of a dedicated SRAM enables hiding the computation inside the hash core in the data transfer. A (from a software perspective) write-only *RX FIFO* buffers the input data and a read-only *TX FIFO* buffers the output data. This enables the co-processor to start absorbing the input data as soon as the first chunk is written

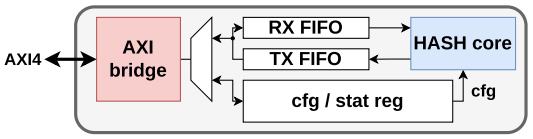



Fig. 4. Loosely coupled architecture using FIFOs.

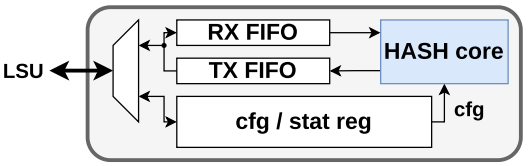



Fig. 5. LSU-coupled architecture using FIFOs.

into the *RX FIFO*. The output data produced by the hash core is written into the *TX FIFO* and the microcontroller can immediately start reading the output. This also reduces the complexity in the module, as no control bits are required anymore. How to properly dimension the FIFOs depends on the underlying hash function (the number of permutation rounds), as, during permutation, no input data can be absorbed and no output data is produced. This is further analyzed in Section 3.5.

### 3.4 LSU Coupled Using FIFOs

Although the usage of FIFOs benefits the performance in general, writing and reading the data via the AXI4 system bus can still pose a notable overhead due to the bus latency. Therefore, the last design option is to integrate the co-processor closer to the RISC-V core and to connect it directly to the core's LSU, as shown in Figure 5. This yields two advantages: First, the latency caused by the system bus is reduced as no bridging is required anymore—it only requires multiplexing between the accelerator's and the remaining address space. Second, the co-processor does not require a complex AXI4 interface anymore, which reduces the area cost. The drawback of this approach is, however, the increased integration complexity. Connecting the module to the LSU requires slightly more knowledge of the microcontroller implementation than knowing a standard interface as, e.g., AXI4.

### 3.5 FIFO Dimensions

For the architectures discussed in Section 3.3 and Section 3.4 the FIFOs' sizes influence the resource cost as well as the accelerator's performance. Both FIFOs have a width of 32 -bit to comply with the word width of the RISC-V core. The *RX FIFOs* should be large enough to buffer incoming data while the hash core is permuting without running full, so that the data transfer is not blocked. That is, the FIFO's depth ($D_{RX}$) depends on the number of permutation rounds $T_{perm}$ of the hash function and the bus latency $t_{bus,wr}$ to perform a write operation. For the *TX FIFO*, it is desirable that whenever the microcontroller wants to read data, the FIFO is not empty. However, it should be able to store enough data that the hash core can finish the next permutation while the FIFO is still being read without running empty. As a consequence, the required depths (in 32 -bit words) of the FIFOs are as follows:

$$D_{RX} \geq \left\lceil \frac{T_{perm}}{t_{bus,wr}} \right\rceil \qquad D_{TX} \geq \left\lceil \frac{T_{perm}}{t_{bus,rd}} \right\rceil. \tag{1}$$

In our design, we have $t_{axi,wr} = 6$, $t_{axi,rd} = 7$, $t_{lsu,wr} = 2$, and $t_{lsu,rd} = 3$, until the data is inside the corresponding FIFO or read from it. With $T_{perm} = 24$ for SHAKE256, we end up with FIFO depths of $D_{RX} = D_{TX} = 4$ for the loosely coupled case and $D_{RX} = 16$ and $D_{TX} = 8$ for the

Table 1. Resource Overhead Including the Hash Core

| Architecture | SHA-3 | | Ascon | |
|---|---|---|---|---|
| | LUTs | FFs | LUTs | FFs |
| baseline no FPU | 14,334 (100%) | 8,720 (100%) | 14,334 (100%) | 8,720 (100%) |
| loosely coupled + SRAM | +8,516 (+59.4%) | +2,477 (+28.4%) | +1,803 (+12.6%) | +1,159 (+13.3%) |
| loosely coupled + FIFOs | +8,760 (+61.1%) | +2,438 (+28.0%) | +1,777 (+12.4%) | +1,130 (+13.0%) |
| LSU coupled | +8,154 (+56.9%) | +1,881 (+21.6%) | +960 (+6.70%) | +523 (+6.01%) |
| tightly coupled | +4,597 (+32.1%) | +1,061 (+12.3%) | +662 (+4.62%) | +19 (+0.22%) |

Maximum frequency of 150 MHz is not affected by accelerators.

LSU-coupled version. Note that although for the LSU-coupled version $D_{RX} = 12$ would meet the requirements, we slightly increased it to stick to a power of two. Due to the small size of the FIFOs, they are inferred as distributed memory (LUTRAM), which has a minimum depth of 32, and thus, the power-of-two ceiling does not increase area cost.

### 3.6 Resource Overhead

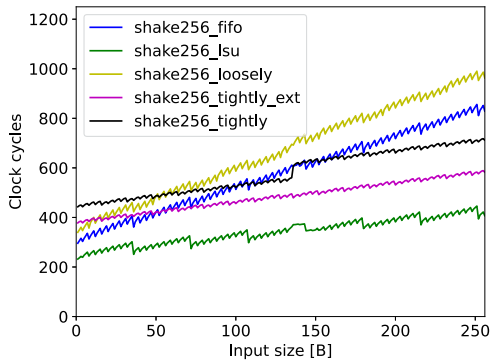
A comparison of the different architectures with respect to resource overhead compared with the plain PULPino design is given in Table 1. To provide a meaningful comparison, we synthesized the whole platform with the different accelerator architectures and only stated their additional overhead. For a discussion on the Ascon cost we refer to Section 5.1. Only stating the resource consumption for the architectures as depicted previously would omit the cost of additional circuitry like bus connections within PULPino's crossbar or the connection to the LSU.

The results highlight the advantage of the tightly coupled approach: Because this architecture only implements the round function in hardware but re-uses the GPR and FPR to store the state, it shows the least overhead. It is worth noting that the tightly coupled's overhead in FFs mainly stems from the $32 \times 32 = 1{,}024$ -bit of the additional FPR. If the corresponding RV32F floating-point extensions are enabled, the baseline platform consumes 20,209 LUTs and 11,535 FFs. Then the additional overhead for the tightly coupled SHA-3 accelerator is only about 2,419 additional LUTs for the Keccak round function and the corresponding instruction. The loosely and LSU-coupled architectures require more logic resources due to bus interfaces, control logic, and the local memory in terms of FIFOs or SRAM. The SRAM overhead for the loosely coupled version is omitted for two reasons. First, the accelerator's SRAM can later be re-used for normal operation of the RISC-V core and thus does not need to be accelerator specific. Second, the SRAM size depends on the specific application—it needs to be big enough to store the maximum amount of input or output data. Nevertheless, a single Xilinx BRAM of our FPGA with the size of 4 kB is sufficient for all our experiments. For SPHINCS+, the minimum required SRAM size would be defined either by the amount of input bytes of the function $T_2$ (as described in Section 2.1) or by the function $H_{msg}$ that hashes a message of arbitrary length. Thus, the minimum size is determined by the maximum length of the inputs for $T_2$ and $H_{msg}$. In contrast to that, the FIFO dimensions are application independent—application independent in the sense that the FIFO dimensions do not depend on the amount of input or output data, but only on the small amount of data that needs to be buffered while the hash core is permuting.

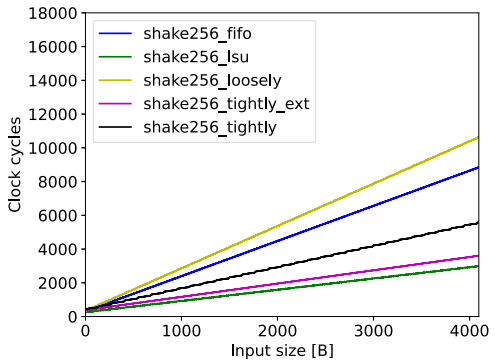
## 4 Performance Evaluation

### 4.1 Architecture Benchmark

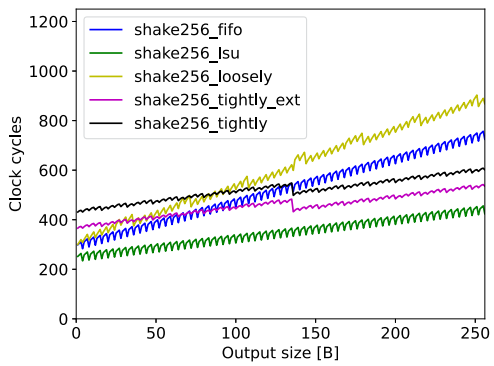
In the following, we compare the performance of the architectures described above. For the benchmark, we consider two scenarios:



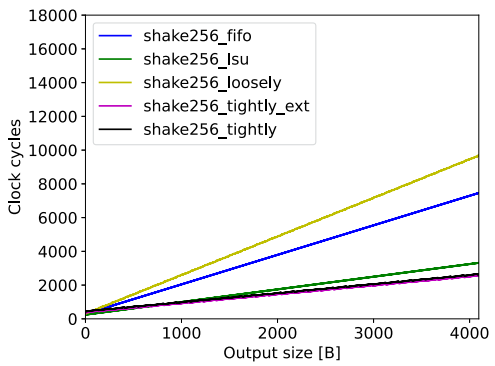

(a) SHAKE256 **Hash** scenario short




(b) SHAKE256 **Hash** scenario long




(c) SHAKE256 **CSPRNG** scenario short




(d) SHAKE256 **CSPRNG** scenario long

Fig. 6. Performance evaluation of the SHAKE256 accelerators in the **Hash** (Figures 6(a) and 6)(b) and **CSPRNG** (Figures 6(c) and 6(d)) scenario.

In the Hash scenario, we benchmark the performance for a fixed output length and vary the input length that is absorbed. Concretely, we set the output length to 32 B and vary the input lengths up to 4 kB. This reflects a use-case, where an arbitrarily sized input is hashed to a fixed-size digest.

In the **Cryptographically Secure Pseudo-Random Number Generator (CSPRNG)** scenario, the input length is fixed to 16 B and the output length varies up to 4 kB. This scenario typically occurs in PQC schemes when a small seed is extended to provide pseudo-random data that is, for instance, used to sample variables. For both scenarios we consider SHAKE256, which can be used for arbitrary input and output lengths and is used within SPHINCS+.

The results for both scenarios are shown in Figure 6. The LSU- and tightly coupled versions (considering *shake256_tightly* for now) perform best depending on the scenario and amount of data to be processed. To explain this, the data transfer has to be considered. In both cases, the input data is first loaded from memory into the register set, and after computation, the hash output is written back to memory. Yet, the computation is different depending on the architecture: In the tightly coupled case, input blocks $b_i$ must first be loaded into some free GPRs. For the parts of the state's rate that lie within the FPR, the corresponding state word must be moved to the GPR, then absorption (xor) is performed, and afterward, the updated word is written back to the FPR. This requires three instructions for the rate part in the FPR (in our implementation 16 regs × 3 instr),

whereas the rate part in the GPR can be absorbed within one single xor instruction (18 regs $\times$ 1 instr). Therefore, the absorption takes 66 clock cycles per input block; as for SHAKE256, the rate consists of 34 words of 4 B each. After absorption of each input block $b_i$ and squeezing of output blocks $b_o$, the state must be permuted for $T_{perm}$ cycles. Thus, the computation can be modeled as

$$t_{tightly} = b_i(66 + T_{perm}) + b_o T_{perm} + t_{sw}, \tag{2}$$

where $t_{sw}$ denotes a software overhead, e.g., for calling functions. It should be noted that the absorption of the last input block might take less than 66 cycles if it is only partially filled.

For the LSU-coupled and loosely coupled architectures, the input data must be written to and the output data read from the accelerator. However, when using FIFOs, the absorption, squeezing, and subsequent permutations are hidden in the data transfer assuming the FIFOs do not run full or empty. With this assumption, the behavior of the architectures can be modeled as follows:

$$t_{loosely,sram} = t_{store} + (b_i + b_o)T_{perm} + t_{load} + t_{hw} \tag{3}$$

$$t_{loosely,fifo} = t_{store} + t_{load} + t_{hw} \tag{4}$$

$$t_{lsu} = t_{store} + t_{load} + t_{hw}. \tag{5}$$

The variables $t_{store}$ and $t_{load}$ denote the cycles to write data to and read data from the accelerator, and $t_{hw}$ denotes the constant overhead of the hardware drivers.

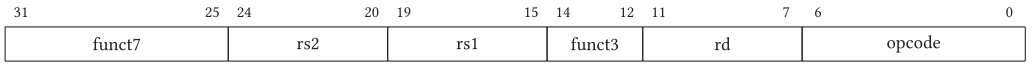
*Implications for Hash Scenario.* For the hash scenario, the $t_{load}$ as well as $b_o$ is negligible, as only a fixed, typically small amount of data is read back from the hash core. For the LSU-coupled approach, Equation (5) yields $t_{lsu} = t_{store} + t_{hw}$. In fact, as the co-processor is connected directly to the LSU, writing to the co-processor only takes a single cycle, and thus, $t_{store} \approx \frac{d_{in}}{4}$ is equal to the amount of input data words, where $d_{in}$ is the input data in bytes. If we neglect the terms $t_{sw}$ and $t_{hw}$, Equation (2) yields $t_{tightly} \approx b_i(66 + T_{perm}) = \lceil \frac{d_{in}}{34\times4} \rceil \times (66 + T_{perm}) \approx d_{in} \frac{90}{136} > \frac{d_{in}}{4}$, such that $t_{tightly} > t_{lsu}$, which confirms the observations of Figures 6(a) and 6(b).

For both loosely coupled scenarios, $t_{load}$ and $t_{store}$ are larger than in the LSU-coupled case. In fact, in our design writing takes six instead of one clock cycle; hence, $t_{loosely,fifo} > t_{lsu}$ according to Equation (4) and (5). Furthermore, as the loosely coupled solution using an SRAM interface cannot hide the absorption and squeezing phase, it is clear that $t_{loosely,sram} > t_{loosely,fifo} > t_{lsu}$. Nevertheless, as the data transfer time for small data sizes is negligible, the computational overhead of the tightly coupled approach exceeds the loosely coupled variants.

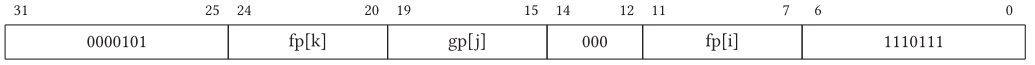
*Implications for CSPRNG Scenario.* If a small seed is absorbed (typically smaller than one block of the Keccak primitive), we can neglect the impact of the terms containing $b_i$, as the absorption phase is negligible. In the LSU-coupled case, we can neglect $t_{store}$ for the same reason. Hence, we end up with $t_{tightly} \approx b_o T_{perm} + t_{sw} = \lfloor \frac{d_{out}}{34\times4} \rfloor 24 + t_{sw} \approx d_{out} \frac{3}{17} + t_{sw}$ and $t_{lsu} \approx t_{load} + t_{hw} = 2\frac{d_{out}}{4} + t_{hw} = \frac{d_{out}}{2} + t_{hw}$, where $d_{out}$ denotes the number of output bytes and loading from the LSU-coupled accelerator requires two cycles. As $\frac{3}{17} < \frac{1}{2}$, the tightly coupled architecture will be faster as soon as the lower computational effort for sufficient amounts of data compensates that $t_{sw} > t_{hw}$. In our experiments this point is reached when squeezing about 952 B of output data. Concerning the two loosely coupled approaches, the same consideration as for the hash scenario holds; i.e., the larger penalty for data transfers dominates the smaller initial overhead compared to the tightly coupled approach from a certain amount of data on.

### 4.2 Extending the RISC-V ISA

The previous section revealed significant performance penalties for the tightly coupled SHA-3 architecture in the Hash scenario due to the need to move some register contents (the ones cor-




(a) Regular R-type instruction




(b) Encoding for the `pq.xor_gp_fp` instruction

Fig. 7. Encoding for the extension of the RISC-V ISA.

responding to the rate of the Keccak state) between the FPR and GPR in the absorption phase of the hash function. This penalty is caused by the fact that the RISC-V ISA does not provide an `xor` instruction between GPRs and FPRs, as in general, there is no direct use for such an operation. However, for our application this would yield a direct speed-up, and thus, we extend the ISA with a corresponding instruction named `pq.xor_gp_fp` as shown in Figure 7.

Figure 7(a) depicts the RISC-V R-type instruction format that performs an operation between two source registers `rs1` and `rs2` and stores the result in the destination register `rd`. For our usecase, we define `rs1` to be a GPR and `rs2` and `rd` to be FPRs as shown in Figure 7(b). Thus, `pq.xor_gp_fp` performs an xor operation between a GPR and an FPR and stores the result again in an FPR. This enables absorbing data into the Keccak state stored in the FPR with the same efficiency as for the part stored in the GPR. The area overhead of the additional instruction is roughly 296 LUTs and 100 FFs. As previously mentioned, it required 66 instructions (16 regs × 3 instr + 18 regs × 1 instr) to absorb a full block of 136 B into the state. With the additional `pq.xor_gp_fp` instruction, this cost reduces to 16 + 18 = 34 instructions, and thus, approximating Equation (2) for the Hash scenario again yields a cost of $t_{tightly} \approx d_{in}\frac{58}{136}$ (previously it was $\approx d_{in}\frac{90}{136}$). While this performance cost is still higher than for the LSU-coupled variant ($\approx \frac{d_{in}}{4}$), it is a significant improvement when absorbing large amounts of data.

To measure the effect, we performed the same benchmark as in Section 4.1. The results are included in Figure 6 denoted as *shake256_tightly_ext*. Compared to the original version denoted as *shake256_tightly*, it clearly shows the advantage of the improved absorption phase with the additional instruction in the Hash scenario. In fact, the performance is much closer to the LSU-coupled version than to the original tightly coupled version. Although in the CSPRNG scenario only a small amount of data is absorbed, it still shows a minor performance gain compared to the original tightly coupled version.

### 4.3 Application to SPHINCS+

The results in Section 4.1 have shown that either the tightly coupled or the LSU-coupled approach shows the best performance. Therefore, we restrict our evaluations for SPHINCS+ to these architectures and present results for the *simple* versions of SPHINCS+ that are considered by NIST [20]. For the *robust* versions, we refer to Appendix A. We consider message sizes of 59 B as in the pqm4 benchmarks [17] and take the round-3 C reference code of the NIST submission as baseline. In the following, all cycle counts are averaged over 50 iterations and the numbers are given in kilo clock cycles ($\times 10^3$).

A comparison of the cycle counts for both architectures is given in Table 2 for the *fast* (f) variant and Table 3 for the *small* variant (s) of all NIST security levels. For comparison, the numbers of the software implementation that does not use hardware acceleration are included. The LSU-coupled architecture shows advantages compared to the tightly coupled alternatives. These results are in

Table 2. Kilo Cycles for SPHINCS+ Fast Variant

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| `128f-simple` | | | |
| shake256_sw | 145,119 | 3,602,835 | 201,209 |
| shake256_lsu | 1,725 | 42,598 | 2,458 |
| shake256_tightly | 2,902 | 71,921 | 4,091 |
| shake256_tightly_ext | 2,608 | 64,617 | 3,682 |
| `192f-simple` | | | |
| shake256_sw | 213,034 | 5,806,444 | 294,981 |
| shake256_lsu | 2,639 | 72,134 | 3,725 |
| shake256_tightly | 4,530 | 123,534 | 6,327 |
| shake256_tightly_ext | 4,100 | 111,819 | 5,731 |
| `256f-simple` | | | |
| shake256_sw | 563,213 | 11,625,520 | 300,496 |
| shake256_lsu | 8,663 | 179,501 | 4,804 |
| shake256_tightly | 13,528 | 280,112 | 7,384 |
| shake256_tightly_ext | 12,395 | 256,725 | 6,779 |

Table 3. Kilo Cycles for SPHINCS+ Small Variant

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| `128s-simple` | | | |
| shake256_sw | 9,288,740 | 70,607,335 | 69,400 |
| shake256_lsu | 109,770 | 837,742 | 852 |
| shake256_tightly | 184,914 | 1,410,992 | 1,419 |
| shake256_tightly_ext | 166,079 | 1,267,837 | 1,278 |
| `192s-simple` | | | |
| shake256_sw | 13,577,843 | 121,983,442 | 100,193 |
| shake256_lsu | 168,459 | 1,538,599 | 1,301 |
| shake256_tightly | 289,052 | 2,617,293 | 2,183 |
| shake256_tightly_ext | 261,535 | 2,370,161 | 1,980 |
| `256s-simple` | | | |
| shake256_sw | 8,974,362 | 106,785,700 | 146,674 |
| shake256_lsu | 137,921 | 1,666,663 | 2,369 |
| shake256_tightly | 216,045 | 2,596,080 | 3,655 |
| shake256_tightly_ext | 197,911 | 2,380,428 | 3,358 |

line with the results observed in the benchmark in Figure 6. Although the additional `pq.xor_fp_gp` instruction yields a decent improvement over the baseline tightly coupled version, the performance is still much lower than for the LSU-coupled architecture. As discussed in Section 2.1, the main hash computations within SPHINCS+ have rather short outputs of only $n$-byte (remember that $n \in \{16, 24, 32\}$), and the LSU-coupled architecture is expected to be faster, as also the previous benchmark showed an advantage of the LSU-coupled version for all input sizes. Compared to the software reference, the LSU-coupled architecture yields speedup factors between ×81 and ×84 for NIST level I (`128f-simple` and `128s-simple`).

### 4.4 Comparison with other Works

Table 4 compares our accelerated design with implementations in the literature in order to point out the speedup obtained by adding a simple, generic SHAKE256 accelerator, as well as the additional improvements obtained when implementing further SPHINCS+ specific optimizations.

The results clearly show a significant performance improvement when offloading the SHAKE256 function to hardware when compared to software implementations running on embedded platforms such as the Cortex-M4 as benchmarked in [17] or on our baseline platform that implements the RISC-V RV32IMC instruction set (*shake256_sw*). It also shows that when taking the interfacing and overall accelerator architecture into account, it even consumes fewer cycles when compared to an AVX2 implementation for an application class processor (Intel Xeon E3-1220) as presented in the SPHINCS+ round-3 submission specification [3]. This is not the case, however, for the tightly coupled architectures as seen when comparing Tables 2 and 4. Nevertheless, the difference is rather small when considering the complexity of an application class processor compared to the PULPino extended with the tightly coupled Keccak accelerator.

While the previous comparison already yields good results, tailoring a co-processor further to the algorithm's needs further increases the speedup due to better usability of the accelerator. Concurrent to our work, [27] implemented further optimizations such as the iterative hashing within SPHINCS+ as well as dedicated data preparation and key-storage within the hardware. As this further lowers the idle time of the co-processor—the load on the SW part for input preparation is reduced—it yields ×9.8, ×8.7, and ×5.6 improvements for keygen, signing, and signature verification, respectively, over our generic LSU-coupled architecture.

Table 4. Clock Cycle Comparison of SPHINCS+ 128f-simple Using SHAKE256 with Other Works

| | Platform | **Keygen** | **Sign** | **Verify** |
|---|---|---|---|---|
| NIST submission [3] | AVX2 | 2,249,444 | 56,933,788 | 3,346,068 |
| PQM4 [17] | Cortex-M4 | 59,759,081 | 1,483,676,214 | 83,065,165 |
| **This** (shake256_sw) | RV32IMC | 145,118,540 | 3,602,834,821 | 201,208,510 |
| **This** (shake256_lsu) | Embedded HW/SW | 1,724,534 | 42,597,665 | 2,457,742 |
| SLotH [27] | Embedded HW/SW | 176,552 | 4,903,978 | 440,636 |

## 5 Deviation from the SPHINCS+ NIST Submission

In this section, we evaluate deviations from the official NIST submission of SPHINCS+. More specifically, we investigate the use of the Ascon suite [9] and evaluate the changes also for round-reduced versions for Ascon and SHAKE256.

### 5.1 Ascon Primitive

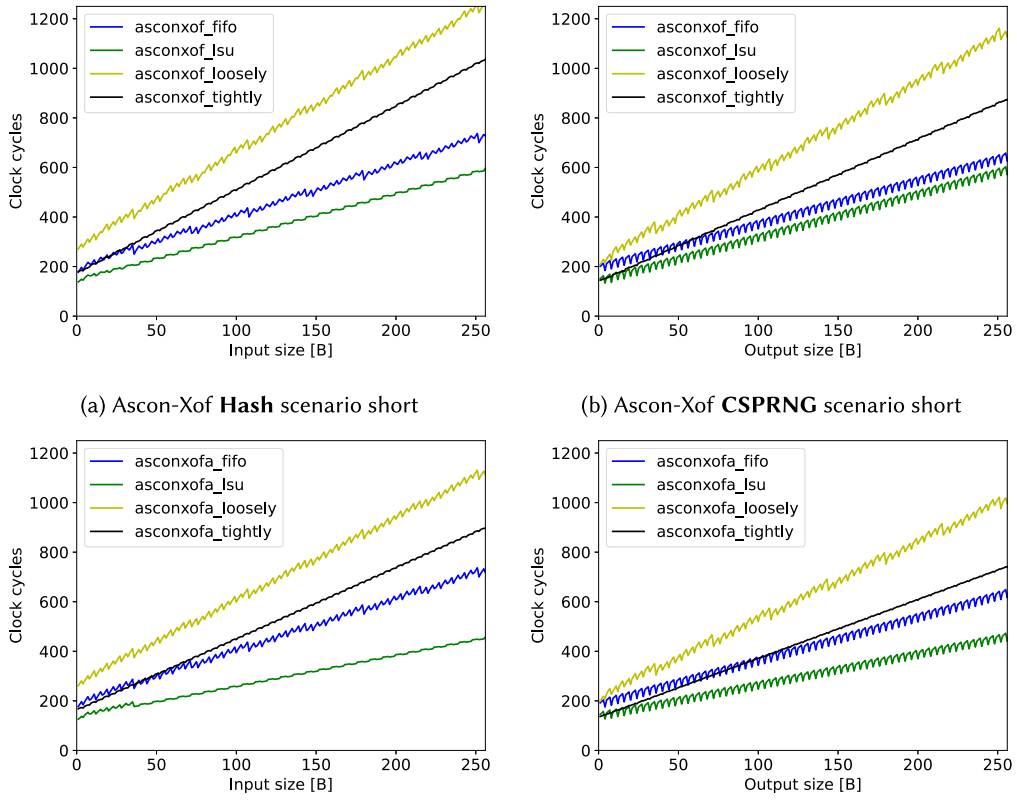
As mentioned previously, SPHINCS+ is a framework that can be instantiated with different primitives for hash functions and pseudo-random number generation. Recently, NIST selected Ascon [9], which also features hash and XOF functions, as the winner of the LWC competition [29].

Just like Keccak, Ascon is based on the sponge construction but consists only of a 320 -bit state and a round function that is applied for 12 rounds. Due to its smaller state and less number of rounds, performance gains could be expected. Yet, it has a rate of $r = 64$ -bit (or 2 words, compared to SHAKE256's 34 words), such that it requires more permutations when large amounts of data are processed. Furthermore, due to the smaller state, Ascon only provides a (second) preimage resistance of 128 -bit and thus reaches only NIST security level I if instantiated within SPHINCS+. For evaluation we developed a small hash core that fits into the architectures described previously. Moreover, we implemented a tightly coupled approach as presented in [28]. In the following, we perform the same analysis as for the SHAKE256 use-case before.

*FIFO Dimensions.* Following Equation (1), the FIFO depths $D_{RX}$ and $D_{TX}$ of the loosely coupled architecture are set to $D_{RX} = D_{TX} = 2$, as Ascon permutes for $T_{perm} = 12$ rounds. For the LSU-coupled version, the same dimensions apply, although the rationale is different: With a write-delay of only two cycles, the microcontroller can provide up to six data words while Ascon is permuting. As the rate thatAscon is only 64 -bit, i.e., two words, data is provided faster than it is processed by the hash core. Thus, setting $D_{RX} = 2$ is sufficient to constantly provide data to the hash core. The same applies for $D_{TX}$: Data is read faster than the hash core can produce, and thus, $D_{TX} = 2$ is sufficient such that the hash core can output a full block and continue permuting the state immediately. As such, the permutation of the Ascon state is the actual bottleneck in the computation.

*Resource Overhead.* Table 1 shows the overhead for the Ascon primitive. Compared to the SHA-3 core, it shows reductions in both the LUT and FF count, mostly due to the smaller state. In particular for the tightly coupled architecture, this overhead is almost negligible. Compared with the tightly coupled SHA-3 version, the reduction in FFs mainly stems from the fact that for the 320 -bit Ascon state, no FPR set is required, as it fits well into the $32 \times 32$ -bit GPR set.

*Architecture Benchmark.* Figures 8(a) and 8(b) show the architecture benchmark using Ascon-Xof. For the CSPRNG scenario (Figure 8(b)), the loosely coupled version with FIFOs is similiarly fast as the LSU-coupled architecture (same slope, only a small offset). This indicates that the Ascon core is the actual bottleneck, but not the bus latency. However, this is not the case for the hash scenario (Figure 8(a)), because the data transmission partially hides the computation in the




(a) Ascon-Xof **Hash** scenario short

(b) Ascon-Xof **CSPRNG** scenario short

(c) Ascon-Xofa **Hash** scenario short

(d) Ascon-Xofa **CSPRNG** scenario short

Fig. 8. Architecture benchmarking with functions from the Ascon suite.

Table 5. Kilo Cycles for SPHINCS+-128f-simple

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| asconxof_sw | 112,543 | 2,793,036 | 159,128 |
| asconxof_lsu | 1,733 | 42,868 | 2,475 |
| asconxof_tightly | 2,298 | 56,896 | 3,282 |

Table 6. Kilo Cycles for SPHINCS+-128s-simple

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| asconxof_sw | 7,203,739 | 54,787,744 | 55,470 |
| asconxof_lsu | 110,308 | 843,462 | 864 |
| asconxof_tightly | 146,232 | 1,116,889 | 1,147 |

absorption phase. For small amounts of data, the tightly coupled version is competitive, but the arithmetic overhead for a tightly coupled option poses a notable drawback in both scenarios when the amount of data increases.

*Application to SPHINCS+.* In Tables 5 and 6 we provide the cycle counts using Ascon-Xof for SPHINCS+. It can be seen that the gap between the two Ascon accelerator architectures is smaller than for the SHAKE256 variants in Tables 2 and 3. While the LSU-coupled versions of SHAKE256 and Ascon are roughly in the same ballpark, the tightly coupled Ascon-Xof design yields better results than its SHAKE256 counterpart, as it does not require the FPR. Compared to the software reference implementation, the LSU-coupled architecture yields speedup factors between ×64 and ×65. Furthermore, a comparison shows that using Ascon-Xof over SHAKE256 already yields a speedup of about 22.5 % for signature generation for the plain software comparison.

### 5.2 Round-reduced Versions

Ascon-Xofa is a round-reduced version of Ascon-Xof that is also part of the Ascon cipher suite. Compared to Ascon-Xof, the first permutation and the permutation after absorbing the last input block remain at 12 rounds; all others are reduced to 8 rounds when using Ascon-Xofa. Reducing the number of rounds within a permutation has also been discussed in the context of the Keccak primitive. There has been a lot of cryptanalysis on the Keccak primitive over the years, but the Keccak primitive has shown to provide a huge security margin. As a result, reducing the number of rounds from 24 to 12 has been formalized under the name of TurboSHAKE256 [7]. In the following, we evaluate the advantages of the round-reduced versions for Ascon-Xof and SHAKE256 and perform the architecture benchmark as well as the impact on SPHINCS+.

*FIFO Dimensions and Resource Overhead.* For TurboSHAKE256, halving the number of rounds per permutation also enables halving the FIFO dimensions of the accelerator. For the Ascon-Xofa variant, however, the *RX FIFO* in the loosely coupled case must be slightly increased to $D_{RX} = 4$.

*Architecture Benchmark.* For the architecture benchmark, our results for TurboSHAKE256 look similar to the ones shown in Figure 6, with only minor offsets, which underlines the dominant impact of the data transfer. As discussed previously, due to the high rate of the Keccak primitive in general, the data transfer poses more of a bottleneck than the actual round permutation, which is mostly hidden. For Ascon-Xofa, this case is slightly different. As already mentioned, the Ascon core is a bottleneck due to its lower rate. This is visible in Figure 8(b), where the slope of the loosely coupled FIFO version and the LSU-coupled architecture are equal. Yet, Figure 8(d) shows that this changes for Ascon-Xofa. The cycle counts are reduced and the slope of the LSU- and loosely coupled FIFO version are different, as the computation within the core is blocking the data transfer less. In addition to that, Figure 8(c) also shows that both the loosely coupled FIFO version and the LSU-coupled version benefit from the round reduction in the hash scenario as well.

*Application to SPHINCS+.* Tables 7 and 8 list the corresponding cycle counts for the round-reduced versions in the context of SPHINCS+. As expected, it shows performance gains for all cases. It is notable, however, that the reduction for the round-reduced version Ascon-Xofa is larger than for TurboSHAKE256 (abbreviated as t_shake256) when taking hardware acceleration into account. This is due to the observation made previously, where the computation inside the hash core posed a bottleneck. Reducing the rounds' speeds up this computation and thus resolves this issue. As a consequence, the Ascon-Xofa design yields the highest performance when instantiated in SPHINCS+ using the LSU-coupled architecture. Compared to the software implementations, the LSU-coupled architecture yields speedup factors of between ×49 and ×51 for TurboSHAKE256 and between ×52 and ×53 for Ascon-Xofa for NIST security level I.

### 5.3 Discussion on Round-reduced Versions for SPHINCS+

Comparing the above results shows a direct performance gain for the round-reduced versions when looking at the pure software implementations. When taking hardware acceleration into account, the performance gain is not as significant as could be expected. To visualize that, Figure 9 depicts these savings for the pure software versions as well as the LSU- and tightly coupled versions (with additional ISA extension) at the example of the signature generation procedure of the SPHINCS+ `128f-simple` variant. In the case of the SHA-3 functions (Figure 9(a)), the software implementations benefit the most from the round reduction. As the permutation function is the main effort in software computation, reducing the rounds by 50 % has a similarly strong impact on the overall performance. However, when employing hardware acceleration, the benefit of round reduction is almost negligible, as the main effort in the computation is not the permutation

Table 7. Kilo Cycles for SPHINCS+ Fast Variant

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| 128f-simple | | | |
| t_shake256_sw | 85,584 | 2,124,535 | 118,879 |
| t_shake256_lsu | 1,684 | 41,594 | 2,401 |
| t_shake256_tightly | 2,848 | 70,574 | 4,014 |
| t_shake256_tightly_ext | 2,554 | 63,270 | 3,605 |
| asconxofa_sw | 82,716 | 2,052,962 | 116,680 |
| asconxofa_lsu | 1,571 | 38,851 | 2,242 |
| asconxofa_tightly | 2,136 | 52,884 | 3,049 |
| 192f-simple | | | |
| t_shake256_sw | 126,040 | 3,435,129 | 175,154 |
| t_shake256_lsu | 2,574 | 70,332 | 3,650 |
| t_shake256_tightly | 4,450 | 121,346 | 6,242 |
| t_shake256_tightly_ext | 4,021 | 109,633 | 5,644 |
| 256f-simple | | | |
| t_shake256_sw | 333,873 | 6,892,927 | 178,042 |
| t_shake256_lsu | 8,508 | 176,300 | 4,722 |
| t_shake256_tightly | 13,319 | 275,794 | 7,272 |
| t_shake256_tightly_ext | 12,186 | 252,407 | 6,667 |

Table 8. Kilo Cycles for SPHINCS+ Small Variant

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| 128s-simple | | | |
| t_shake256_sw | 5,477,698 | 41,640,940 | 40,760 |
| t_shake256_lsu | 106,892 | 815,862 | 828 |
| t_shake256_tightly | 181,436 | 1,384,566 | 1,388 |
| t_shake256_tightly_ext | 162,602 | 1,241,410 | 1,248 |
| asconxofa_sw | 5,294,278 | 40,266,442 | 40,881 |
| asconxofa_lsu | 99,944 | 764,591 | 788 |
| asconxofa_tightly | 135,874 | 1,038,107 | 1,073 |
| 192s-simple | | | |
| t_shake256_sw | 8,009,407 | 71,976,146 | 59,307 |
| t_shake256_lsu | 164,690 | 1,504,662 | 1,278 |
| t_shake256_tightly | 283,971 | 2,571,655 | 2,153 |
| t_shake256_tightly_ext | 256,454 | 2,324,524 | 1,949 |
| 256s-simple | | | |
| t_shake256_sw | 5,304,720 | 63,147,585 | 86,923 |
| t_shake256_lsu | 135,171 | 1,633,796 | 2,331 |
| t_shake256_tightly | 212,696 | 2,556,251 | 3,610 |
| t_shake256_tightly_ext | 194,563 | 2,340,601 | 3,313 |

anymore, but rather the data transfer and, in the case of the tightly coupled version, the remaining instructions required for absorption, byte counters, stack operations, and so forth.

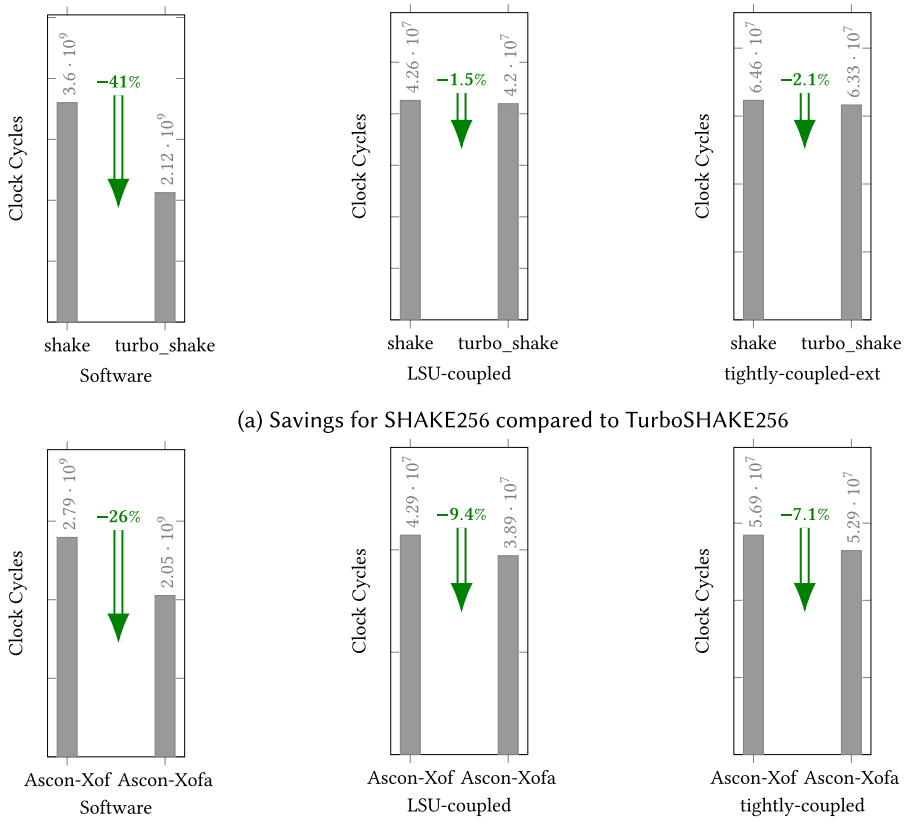
When looking at the case for Ascon (Figure 9(b)), the round reduction also benefits the hardware-accelerated versions. This can be explained by the observation made previously that due to Ascon's smaller rate, it must permute more often and partially slows down the data transfer speed in the LSU-coupled case. In the tightly coupled case, the overall software driver requires fewer instructions, and thus, the permutation itself makes up a larger part of the overall computational effort compared to the SHAKE256 version. As a consequence, the tightly coupled Ascon variant benefits more from a round reduction (relatively speaking) than the SHAKE256 variant.

Taking these observations into account implies that additional design changes like unrolling of multiple rounds (i.e., computing multiple rounds in a single clock cycle) would not yield a significant performance improvement for SHAKE256 hardware acceleration on HW/SW co-designs. While the Ascon variants could further benefit from unrolling the rounds, the performance gain is expected to decrease as soon as the point is reached where the permutation is not a significant factor anymore (as it is for SHAKE256).

## 6 Application to Other PQC Schemes

In this section, we extend the analysis to additional PQC schemes. To be precise, we investigate the behavior of SHA-3 hardware acceleration for the signature schemes CRYSTALS-Dilithium [10] and Falcon [11], as well as the KEM CRYSTALS-Kyber [8]. All of them have been selected by NIST for standardization under the name of ML-KEM [22] (Kyber), ML-DSA [21] (Dilithium), and SLH-DSA [23] (SPHINCS+) with small modifications.

Just as for SPHINCS+, we consider the C reference implementation of round 3 of the NIST standardization as software reference and perform 10,000 measurements for a 59 B message per algorithm. For the accelerated versions we adapted our driver functions to be compatible with the incremental SHA-3 API used in the reference code. As the additional PQCs are from the class of lattice-based algorithms, the instantiated symmetric primitive is mostly used as CSPRNG. Previous benchmarks have shown that SHA-3 performs better for large amounts of squeezed data compared




(a) Savings for SHAKE256 compared to TurboSHAKE256

(b) Savings for Ascon-Xof compared to Ascon-Xofa

Fig. 9. Relative savings for SPHINCS+ signature generation (128f-simple) when using the round-reduced versions of the primitives.

to the Ascon counterparts, and thus we consider only the SHA-3 accelerators in the following. In the following we restrict the evaluation to the improved tightly coupled and LSU-coupled accelerator, as they have shown the best performance in general.

The benchmark results for Dilithium and Falcon are shown in Table 9 with the standard SHA-3 functions instantiated. We note that Dilithium and Kyber instantiate SHAKE128 to generate a public matrix $\boldsymbol{A}$. We respected that in our accelerated versions, although we denote the measurement run as SHAKE256 for better readability in Table 9. Table 10 lists the results for the round-reduced TurboSHAKE functions.

Comparing the numbers for the different accelerators, it is observable that the difference between the LSU- and tightly coupled architectures is rather negligible. In fact, for the Falcon parameters they differ by less than 0.5 %. For the Dilithium parameter sets, this difference is slightly larger but still below 4 %. The numbers are mostly in favor of the tightly coupled solution, although there are exceptions, such as, for example, the key generation and signature verification procedures for Dilithium5. For the Falcon signature generation process, the SHA-3 hardware acceleration has little impact as the time the algorithm spends within the hash function is below 1 % according to the pqm4 benchmarks.[4]

[4]https://github.com/mupq/pqm4/blob/master/benchmarks.md, commit: 918f379.

Table 9. Kilo Cycles for CRYSTALS-Dilithium and Falcon Using Standard SHAKE

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| Falcon512 | | | |
| shake256_sw | 286,426 | 70,381 | 585 |
| shake256_lsu | 213,886 | 70,122 | 351 |
| shake256_tightly | 212,996 | 70,301 | 350 |
| Falcon1024 | | | |
| shake256_sw | 750,812 | 153,913 | 1,206 |
| shake256_lsu | 630,258 | 153,419 | 745 |
| shake256_tightly | 627,305 | 153,812 | 741 |
| Dilithium2 | | | |
| shake256_sw | 3,866 | 12,915 | 4,122 |
| shake256_lsu | 885 | 6,880 | 1,165 |
| shake256_tightly | 870 | 6,741 | 1,156 |
| Dilithium3 | | | |
| shake256_sw | 6,930 | 20,544 | 6,883 |
| shake256_lsu | 1,351 | 11,318 | 1,745 |
| shake256_tightly | 1,302 | 11,175 | 1,701 |
| Dilithium5 | | | |
| shake256_sw | 11,486 | 26,315 | 11,748 |
| shake256_lsu | 2,069 | 13,091 | 2,561 |
| shake256_tightly | 2,088 | 12,984 | 2,576 |

Table 10. Kilo Cycles for CRYSTALS-Dilithium and Falcon Using TurboSHAKE

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| Falcon512 | | | |
| t_shake256_sw | 252,868 | 70,256 | 472 |
| t_shake256_lsu | 214,166 | 70,124 | 351 |
| t_shake256_tightly | 213,244 | 70,302 | 349 |
| Falcon1024 | | | |
| t_shake256_sw | 698,012 | 153,681 | 987 |
| t_shake256_lsu | 632,931 | 153,420 | 745 |
| t_shake256_tightly | 629,925 | 153,813 | 741 |
| Dilithium2 | | | |
| t_shake256_sw | 2,564 | 10,268 | 2,820 |
| t_shake256_lsu | 885 | 6,903 | 1,165 |
| t_shake256_tightly | 869 | 6,800 | 1,155 |
| Dilithium3 | | | |
| t_shake256_sw | 4,458 | 16,375 | 4,594 |
| t_shake256_lsu | 1,351 | 11,335 | 1,745 |
| t_shake256_tightly | 1,300 | 11,169 | 1,699 |
| Dilithium5 | | | |
| t_shake256_sw | 7,330 | 20,397 | 7,671 |
| t_shake256_lsu | 2,069 | 13,130 | 2,561 |
| t_shake256_tightly | 2,085 | 12,907 | 2,572 |

Comparing Table 9 and Table 10 shows the direct benefits of using TurboSHAKE instead of SHAKE for the software versions. As previously noted in the benchmarks for SPHINCS+, there is no direct improvement for the hardware accelerated versions. There are basically two observations to be made: First, for functions like signature verification, where the output of the hash function has no impact on the runtime of the algorithm, the performance improvements disappear already in the rounding toward kilo cycles ($\times 10^3$). Second, for key and signature generation, most cycle counts seem to even increase for the TurboSHAKE measurements. Although this is not possible for the runtime of the hash function itself, the fact that the outputs of SHAKE and TurboSHAKE differ, has an impact on the runtime of, e.g., the rejection sampling process within the algorithm. As a consequence, changing the hash output can yield slower results for single measurements. Yet this effect should disappear for a higher number of experiments.

In Table 11 and Table 12, we show the corresponding results for the KEM Kyber in cycles (in contrast to kilo cycles for the signature algorithms). Throughout all parameter sets, the LSU-coupled version is faster than the tightly coupled version. The performance difference, however, is again negligible (less than 0.5 %). Also, the observations made previously for the TurboSHAKE versions hold: while it yields direct performance gains in software, it is negligible for hardware acceleration.

## 7 Area, Power, and Energy Estimations for a 22 nm ASIC Design

### 7.1 Area Results

In this section, we discuss the resource overhead of the different accelerator architectures for an ASIC design on a 22 nm technology, namely the Globalfoundries 22FDSOI process. All results were obtained using Cadence Genus 21.17 and Low Threshold Voltage cells from Synopsys for the given

Table 11. Cycle Count for CRYSTALS-Kyber Using SHAKE256

| Architecture | Keygen | Encaps | Decaps |
|---|---|---|---|
| Kyber512 | | | |
| shake256_sw | 1,144,653 | 1,489,763 | 1,465,204 |
| shake256_lsu | 327,542 | 457,337 | 642,028 |
| shake256_tightly | 328,496 | 458,907 | 643,292 |
| Kyber768 | | | |
| shake256_sw | 1,855,586 | 2,377,552 | 2,323,909 |
| shake256_lsu | 550,632 | 713,484 | 956,781 |
| shake256_tightly | 552,291 | 715,715 | 958,688 |
| Kyber1024 | | | |
| shake256_sw | 2,918,755 | 3,560,411 | 3,478,511 |
| shake256_lsu | 826,400 | 1,025,291 | 1,327,952 |
| shake256_tightly | 828,801 | 1,028,323 | 1,330,613 |

Table 12. Cycle Count for CRYSTALS-Kyber Using TurboSHAKE256

| Architecture | Keygen | Encaps | Decaps |
|---|---|---|---|
| Kyber512 | | | |
| t_shake256_sw | 789,173 | 1,042,226 | 1,109,724 |
| t_shake256_lsu | 327,481 | 457,237 | 641,948 |
| t_shake256_tightly | 328,171 | 458,498 | 642,967 |
| Kyber768 | | | |
| t_shake256_sw | 1,289,208 | 1,653,361 | 1,731,228 |
| t_shake256_lsu | 550,544 | 713,341 | 956,660 |
| t_shake256_tightly | 551,774 | 715,054 | 958,147 |
| Kyber1024 | | | |
| t_shake256_sw | 2,009,695 | 2,454,087 | 2,543,150 |
| t_shake256_lsu | 826,288 | 1,025,137 | 1,327,820 |
| t_shake256_tightly | 827,970 | 1,027,312 | 1,329,758 |

Table 13. Resource Overhead for Globalfoundries 22FDSOI Technology at a Frequency of 800 MHz

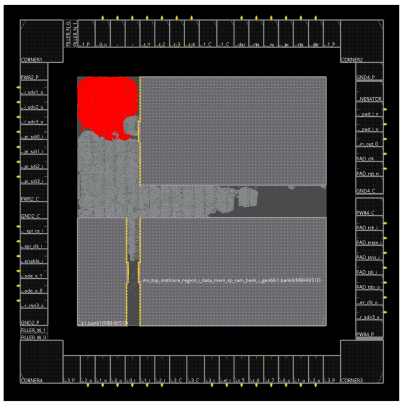
| Architecture | SHA-3 | | Ascon | |
|---|---|---|---|---|
| | [$\mu m^2$] | kGE | [$\mu m^2$] | kGE |
| baseline no FPU | 544,685 (100%) | 389.7 (100%) | 544,685 (100%) | 389.7 (100%) |
| ↳ memories | 334,819 (61.5%) | 239.5 (61.5%) | 344,819 (61.5%) | 239.5 (61.5%) |
| LSU coupled | +19,060 (+3.5%) | +13.6 (+3.5%) | +4,157 (+0.76%) | +2.96 (+0.76%) |
| tightly coupled | +14,058 (+2.6%) | +10.0 (+2.6%) | +2,451 (+0.45%) | +1.74 (+0.45%) |

process. All designs run at a core frequency of 800 MHz, where the maximum frequency of the design is not affected by the accelerators but limited by the memory connections.

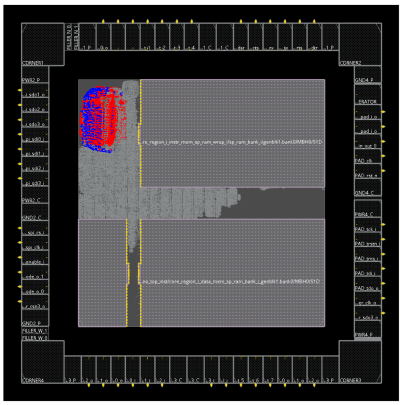
Table 13 shows the area results of the designs on a post-synthesis basis with an additional area estimation for the routing (estimated by the synthesis tool). Just as for the FPGA-specific results, the ASIC metrics are in line with the fact that the tightly coupled solution has a much smaller footprint compared to the LSU-coupled version, as well as the area advantage of using Ascon over SHA-3 for appropriate parameter sets. Nevertheless, it shows that the overall size is mostly determined by the memory area required to run all the investigated algorithms: about 60 % of the area is consumed by memories, as we employed about 128 kB of instruction RAM and 160 kB of data RAM. To illustrate the different designs, Figure 10 depicts the floorplan of the placed ASIC designs with the accelerators highlighted in red. In Figure 10(b) we additionally highlighted the FPR in blue, as its presence is required by the accelerator.

### 7.2 Power and Energy Consumption

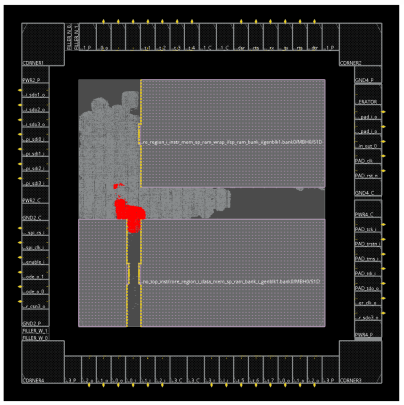
In the following, we analyze the impact of the accelerators on the power and energy consumption for the PQC schemes discussed above. All results are obtained using Cadence Joules 21.10 on the synthesis results presented above for a single key and signature generation and verification procedure (respectively, encapsulation and decapsulation for Kyber). The designs were evaluated at 800 MHz using toggle count dumps from Cadence Xcelium in the `.saif` format. When talking about *power* in the following, we refer to the total power consumption averaged over the runtime of the procedures, e.g., the average power consumption for one key generation. When talking about *energy*, we refer to the total energy consumption of design required for the corresponding



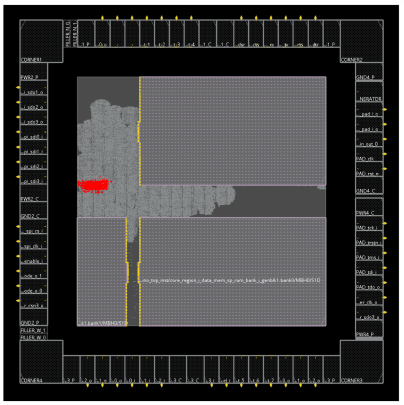
(a) Design with LSU-coupled SHA-3 accelerator.



(b) Design with tightly-coupled SHA-3 accelerator.



(c) Design with LSU-coupled Ascon accelerator.



(d) Design with tightly-coupled Ascon accelerator.

Fig. 10. ASIC designs with the accelerators highlighted in red, FPR for tightly coupled SHA-3 in blue.

procedure. For all the designs, static leakage was about 1 mW with a variation of maximum 3 %, depending on the accelerator architecture. As this is relatively low compared to the variance in dynamic power consumption, we only discuss total power consumption in the following.

The average power consumption of the designs is depicted in Figure 11 at the example of a SPHINCS+ signature verification (128f-simple parameter) for normal and round-reduced accelerator versions. It shows the lowest power consumption for the baseline design without any accelerator (SHAKE256 software execution). This is to be expected as it has the smallest area. Consequently, it has the lowest amount of switching cells per clock cycles and, thus, the lowest power consumption on average. In general, it holds that the larger the area of the design, the larger the power consumption. This general observation holds also for the other algorithms, although their average power consumption slightly differs (mainly depending on the usage of the accelerators).

Given the average power consumption, we can compute the energy consumption for the desired procedures in the following by multiplying the power consumption with the corresponding runtime. For SPHINCS+ we consider only the signature verification in the following discussion, as collecting simulation traces takes a significant amount of time due to its large runtime. However, due to the inner working structure of SPHINCS+, the results should also hold for key and signature generation when scaled appropriately. The energy consumption of SPHINCS+ (128f-

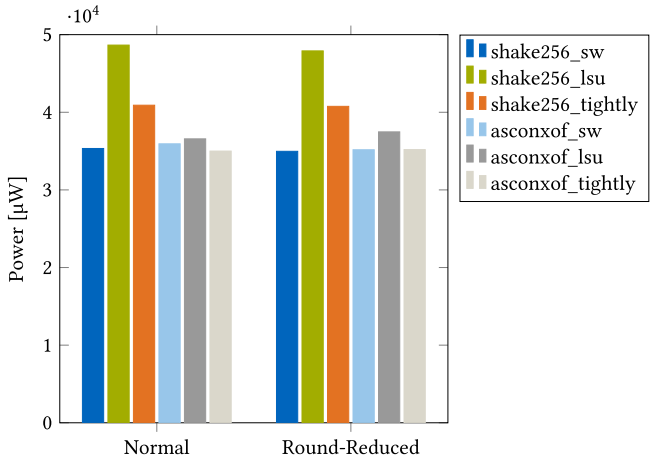
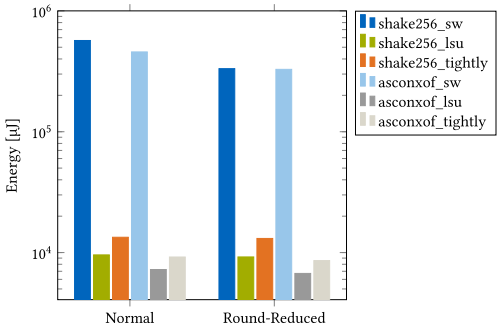
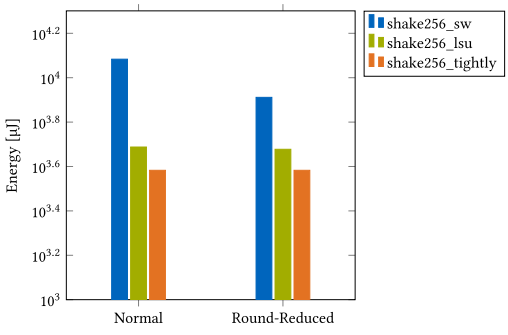


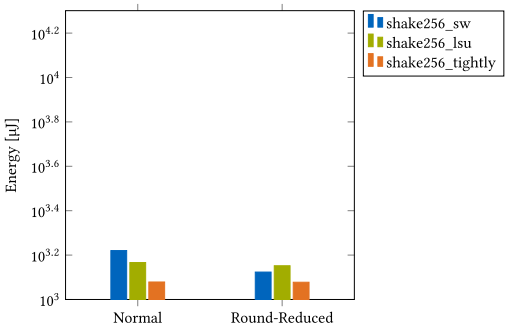

Fig. 11. Average power consumption of a SPHINCS+ (128f-simple) signature verification.



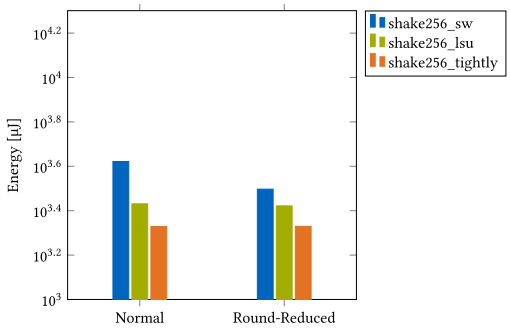

(a) SPHINCS+ (128f-simple) verification.




(b) Dilithium2 verification.




(c) Falcon512 verification




(d) Kyber512 decapsulation.

Fig. 12. Energy consumption of the accelerated designs for investigated algorithms.

simple), Dilithium, and Falcon (all signature verification) as well as Kyber decapsulation is shown in Figure 12 for the lowest security level. For numbers of additional parameters and functions, we refer to Appendix B.

*Energy Consumption for SPHINCS+.* For SPHINCS+, the results show a clear advantage of the LSU-coupled accelerator for SHAKE256 with a reduction of 98 % compared to the pure software

version and 28 % compared to the tightly coupled architecture. Although the LSU-coupled architecture results in higher power consumption as seen previously, the speed advantage as stated in previous benchmarks compensates for this overhead. For the software version, the energy consumption clearly follows the reduction in cycle counts (about 41 %) when comparing TurboSHAKE256 with SHAKE256. For the same reason the savings for the round-reduced versions are smaller for hardware acceleration. In fact, when using TurboSHAKE256 (respectively, Ascon-Xofa) instead of the normal versions, the savings are less than 10 %. When comparing SHAKE256 with Ascon-Xof, both hardware architectures yield an energy advantage in favor of Ascon-Xof as both the cycle counts and area cost for Ascon-Xof are lower.

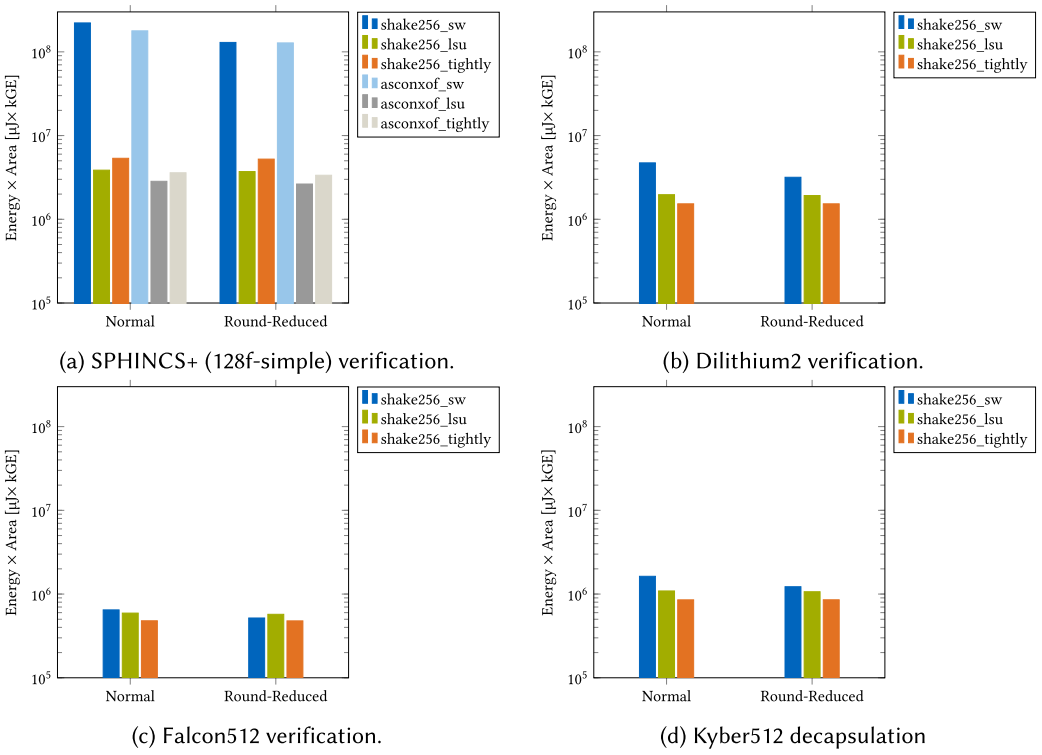
*Energy Consumption for Dilithium and Kyber.* The energy consumption for Dilithium verification and Kyber decapsulation is shown in Figure 12(b) and Figure 12(d). Compared to SPHINCS+, they exhibit a different behavior. On the one hand, the difference between software and hardware accelerated versions is smaller, only about 60 % for the LSU-coupled and 69 % for the tightly coupled version for Dilithium. This is caused by the fact that both algorithms spend a smaller percentage of their runtime in the corresponding hash functions. Consequently, an improvement in runtime has a smaller effect, while at the same time, the designs with hardware acceleration have a higher power consumption. On the other hand, the tightly coupled architecture has a smaller energy consumption compared to the LSU-coupled architecture. This is due to the larger power consumption of the LSU-coupled design, while both accelerators exhibit approximately the same performance improvement in number of clock cycles.

Also, these results hold for the other procedures and parameter sets of the two algorithms. The corresponding data can be found in Appendix B.

*Energy Consumption for Falcon.* The energy consumption for the signature verification of Falcon is shown in Figure 12(c). Due to its good performance, the energy consumption is significantly lower compared to the other schemes. However, this also results in a relatively low benefit when using hardware acceleration, i.e., only about a 12 % improvement for the LSU-coupled and 28 % for the tightly coupled architecture. When considering TurboSHAKE256, the LSU-coupled accelerator performs even worse than the software version, as the software version receives a performance gain from the round reduction, while the accelerated versions remain approximately at the same number of clock cycles as shown previously. Nevertheless, the tightly coupled solution yields benefits in both performance and energy consumption.

The energy results for Falcon's signature verification cannot be generalized to its other procedures as their hash percentage differs significantly. While for the key generation the tightly coupled SHAKE256 accelerator yields better results than the software version, it has a larger energy demand for signature generation. Also, for the round-reduced TurboSHAKE256 versions, the software implementation has the lowest energy consumption for both key and signature generation as it benefits the most from round reduction. The energy and power consumption of Falcon's remaining functions can be found in detail in Appendix B.

*Relation of Area and Required Energy.* As a final evaluation, we put area (in kGE) and required energy for the algorithms into relation and compute the $area \times energy$ product as a general performance metric (in which lower is better). The corresponding results are shown in Figure 13. Taking this metric into consideration, one can draw three conlusions: (1) for algorithms (like lattice-based ones) that use the underlying symmetric primitive mainly as CSPRNG, the tightly coupled variant yields probably the most efficient solution for hardware acceleration, as the squeezed data does not need to be moved between accelerators and memory; (2) for algorithms with a huge amount of hashing or a long absorption phase, an architecture like the LSU-coupled one that allows to




(a) SPHINCS+ (128f-simple) verification.

(b) Dilithium2 verification.

(c) Falcon512 verification.

(d) Kyber512 decapsulation

Fig. 13. Area × energy product of designs for the investigated algorithms.

hide permutations, padding, and so forth in the data transfer yields better overall results; and (3) while for software implementations the round reduction of the hash functions yields direct improvements, these benefits are rather small when considering hardware acceleration.

## 8 Conclusion

Flexible hardware architectures for the acceleration of hash functions benefit a wide range of applications as they can be used for classical hash computation, for pseudo-random number generation, or as subcomponents for post-quantum cryptography. While hardware acceleration for hash functions is an essential step to improve the performance of post-quantum cryptography, the efficiency of such accelerators depends on their specific use-case—whether they are used to compute a hash digest or generate pseudo-random data. It is furthermore essential to take the communication overhead of the accelerator's interface into account and employ a fast interfacing strategy to minimize the idle time of the accelerator.

In this work, we have shown that for a hash-based signature scheme like SPHINCS+, accelerator architectures that can hide the absorption phase in the data transfer are best suited. That is, connecting a SHAKE256 accelerator directly to the LSU showed more than 30 % performance improvements over a tightly coupled architecture and more than 80 times better performance compared to the plain software reference.

In contrast to that, we have shown that for the lattice-based schemes, Dilithium, Falcon, and Kyber, that mainly use SHAKE256 to generate large amounts of pseudo-randomness, the tightly coupled architecture poses the overall better solution. Requiring less area at about the same performance, it consumes about 20 % less energy than the LSU-coupled architecture.

We furthermore investigated the application of the round-reduced TurboSHAKE256 for post-quantum schemes and also the usage of Ascon as underlying primitive for SPHINCS+. While TurboSHAKE256 yields considerable performance gains in software, the benefit when using hardware acceleration is negligible as the permutation itself poses no bottleneck in hardware. Using Ascon-Xofa over TurboSHAKE256 for SPHINCS+, however, yields an additional 7 % speedup and at the same time significantly reduces the resource consumption of the accelerator.

*Future Directions.* Exploring the efficiency of generic hardware acceleration for hash functions provides insight into the performance profile of various PQC schemes when considering the purpose of the hash function. In this sense, profiling the suitability of the hash architectures for the new on-ramp signatures is a future direction. Several on-ramp signatures apply the Fiat-Shamir transform on a Zero-Knowledge protocol and thus heavily use the hash function for both random number generation and hashing to compute, e.g., commitments. Therefore, their hash usage is a mixture of the two application scenarios investigated in this work.

In addition to that, tailoring the accelerator to a specific algorithm yields further speedups and can significantly improve the overall efficiency. For instance, Saarinen presented a co-processor for SLH-DSA (the standardized variant of SPHINCS+) that, on top of generic hashing, provides further support for input preparation, iterative hash chaining, and storing of key material directly in the co-processor in [27]. These measures yield significant improvements (in terms of both power/energy and performance) due to higher workload for the accelerator at low additional cost. Furthermore, schemes like Kyber and Dilithium use the generated pseudo-randomness mainly for rejection sampling to obtain polynomials with coefficients in a specified range and would consequently benefit from an accelerator that also provides direct support for the rejection sampling procedure in order to prevent excessive data movements. Since the standards for the selected algorithms are now to be finalized as FIPS 203 [22] (ML-KEM), FIPS 204 [21] (ML-DSA), and FIPS 205 [23] (SLH-DSA), adapting an accelerator to the particular needs of the algorithms (as done in, e.g., [27]) certainly promises further efficiency improvements.

Another important aspect when implementing cryptographic functions is the need for protection against passive side-channel attacks as well as active fault attacks. Countermeasures against these attacks are usually very costly either in terms of latency and/or in terms of area. Exploring the efficiency of architectures for protected hash accelerators is therefore considered as future work to shed light on the performance metrics of different system design options.

## Appendices

### A SPHINCS+ Performance Evaluation for "Robust" Parameter Sets

Tables 14 and 15 list performance results for the robust versions of SPHINCS+ for both normal and round-reduced versions of SHA-3 and Ascon.

Table 14. Kilo Cycles for SPHINCS+ Robust Variant with Normal Primitives

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| 128f-robust | | | |
| shake256_sw | 281,604 | 6,922,171 | 404,171 |
| shake256_lsu | 3,036 | 74,649 | 4,378 |
| shake256_tightly | 4,787 | 117,787 | 6,868 |
| shake256_tightly_ext | 4,219 | 103,815 | 6,055 |
| asconxof_sw | 199,831 | 4,924,005 | 290,483 |
| asconxof_lsu | 2,575 | 63,391 | 3,733 |
| asconxof_tightly | 3,456 | 85,114 | 5,005 |
| 192f-robust | | | |
| shake256_sw | 411,443 | 10,977,854 | 586,116 |
| shake256_lsu | 4,652 | 125,449 | 6,666 |
| shake256_tightly | 7,406 | 198,915 | 10,535 |
| shake256_tightly_ext | 6,578 | 176,790 | 9,360 |
| 256f-robust | | | |
| shake256_sw | 1,082,127 | 21,853,781 | 594,988 |
| shake256_lsu | 14,150 | 289,552 | 7,870 |
| shake256_tightly | 21,012 | 428,612 | 11,631 |
| shake256_tightly_ext | 18,831 | 384,539 | 10,439 |
| 128s-robust | | | |
| shake256_sw | 18,025,859 | 135,522,303 | 137,482 |
| shake256_lsu | 193,479 | 1,462,957 | 1,499 |
| shake256_tightly | 305,575 | 2,307,961 | 2,350 |
| shake256_tightly_ext | 269,200 | 2,034,449 | 2,073 |
| asconxof_sw | 12,791,730 | 96,538,066 | 100,524 |
| asconxof_lsu | 163,943 | 1,243,324 | 1,293 |
| asconxof_tightly | 220,343 | 1,669,499 | 1,734 |
| 192s-robust | | | |
| shake256_sw | 26,225,007 | 228,378,093 | 199,947 |
| shake256_lsu | 296,436 | 2,640,417 | 2,303 |
| shake256_tightly | 472,001 | 4,169,307 | 3,634 |
| shake256_tightly_ext | 418,953 | 3,707,085 | 3,231 |
| 256s-robust | | | |
| shake256_sw | 17,387,537 | 197,491,089 | 294,022 |
| shake256_lsu | 226,526 | 2,655,492 | 3,923 |
| shake256_tightly | 335,788 | 3,908,727 | 5,756 |
| shake256_tightly_ext | 300,895 | 3,511,878 | 5,169 |

Table 15. Kilo Cycles for SPHINCS+ Robust Variant with Round-reduced Primitives

| Architecture | Keygen | Sign | Verify |
|---|---|---|---|
| 128f-robust | | | |
| t_shake256_sw | 166,218 | 4,087,417 | 237,077 |
| t_shake256_lsu | 2,958 | 72,745 | 4,242 |
| t_shake256_tightly | 4,682 | 115,238 | 6,678 |
| t_shake256_tightly_ext | 4,114 | 101,262 | 5,869 |
| asconxofa_sw | 147,965 | 3,644,704 | 215,432 |
| asconxofa_lsu | 2,336 | 57,477 | 3,395 |
| asconxofa_tightly | 3,173 | 78,144 | 4,610 |
| 192f-robust | | | |
| t_shake256_sw | 242,926 | 6,480,267 | 348,422 |
| t_shake256_lsu | 4,538 | 122,351 | 6,553 |
| t_shake256_tightly | 7,252 | 194,734 | 10,393 |
| t_shake256_tightly_ext | 6,424 | 172,618 | 9,208 |
| 256f-robust | | | |
| t_shake256_sw | 637,755 | 12,881,170 | 352,142 |
| t_shake256_lsu | 13,817 | 282,776 | 7,728 |
| t_shake256_tightly | 20,606 | 420,369 | 11,465 |
| t_shake256_tightly_ext | 18,425 | 376,302 | 10,267 |
| 128s-robust | | | |
| t_shake256_sw | 10,639,442 | 79,995,694 | 81,284 |
| t_shake256_lsu | 188,460 | 1,425,213 | 1,464 |
| t_shake256_tightly | 298,835 | 2,257,290 | 2,303 |
| t_shake256_tightly_ext | 262,460 | 1,983,778 | 2,026 |
| asconxofa_sw | 9,471,374 | 71,460,118 | 74,713 |
| asconxofa_lsu | 148,652 | 1,127,460 | 1,178 |
| asconxofa_tightly | 202,286 | 1,533,103 | 1,601 |
| 192s-robust | | | |
| t_shake256_sw | 15,438,267 | 134,483,204 | 117,396 |
| t_shake256_lsu | 288,321 | 2,569,633 | 2,237 |
| t_shake256_tightly | 462,152 | 4,083,594 | 3,551 |
| t_shake256_tightly_ext | 409,104 | 3,621,372 | 3,149 |
| 256s-robust | | | |
| t_shake256_sw | 10,277,186 | 116,794,591 | 174,009 |
| t_shake256_lsu | 221,730 | 2,600,825 | 3,850 |
| t_shake256_tightly | 329,300 | 3,835,082 | 5,656 |
| t_shake256_tightly_ext | 294,407 | 3,438,235 | 5,069 |

## B Power and Energy Evaluation

The following Tables 16 throughout 35 list the power and energy consumption of the investigated algorithms for their key generation and signature generation and verification (respectively, en- and decapsulation for Kyber). For SPHINCS+ we restricted the measurements to the lowest security level and signature verification due to excessive simulation time.

Table 16. Power and Energy Consumption of SPHINCS+ Signature Verification for Normal Hash Functions

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| `128f-simple` | | | |
| shake256_sw | 35,337 | 201,209 | 568,809 |
| shake256_lsu | 48,657 | 2,458 | 9,567 |
| shake256_tightly | 40,913 | 4,091 | 13,390 |
| asconxof_sw | 35,937 | 159,128 | 457,486 |
| asconxof_lsu | 36,577 | 2,475 | 7,242 |
| asconxof_tightly | 35,004 | 3,282 | 9,190 |
| `128s-simple` | | | |
| shake256_sw | 35,340 | 69,400 | 196,207 |
| shake256_lsu | 48,704 | 852 | 3,319 |
| shake256_tightly | 40,926 | 1,419 | 4,645 |
| asconxof_lsu | 36,525 | 864 | 2,524 |
| asconxof_tightly | 35,135 | 1,147 | 3,223 |

Table 17. Power and Energy Consumption of SPHINCS+ Signature Verification for Round-reduced Hash Functions

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| `128f-simple` | | | |
| t_shake256_sw | 34,973 | 118,879 | 332,604 |
| t_shake256_lsu | 47,909 | 2,401 | 9,202 |
| t_shake256_tightly | 40,765 | 4,014 | 13,090 |
| asconxofa_sw | 35,171 | 116,680 | 328,300 |
| asconxofa_lsu | 37,476 | 2,242 | 6,721 |
| asconxofa_tightly | 35,193 | 3,049 | 8,584 |
| `128s-simple` | | | |
| t_shake256_sw | 34,999 | 40,760 | 114,124 |
| t_shake256_lsu | 47,869 | 828 | 3,170 |
| t_shake256_tightly | 40,860 | 1,388 | 4,537 |
| asconxofa_lsu | 37,460 | 788 | 2,361 |
| asconxofa_tightly | 35,323 | 1,073 | 3,032 |

Table 18. Power and Energy Consumption of CRYSTALS-Dilithium Keygen Using SHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| `Dilithium2` | | | |
| shake256_sw | 36,102 | 3,866 | 11,165 |
| shake256_lsu | 52,103 | 885 | 3,688 |
| shake256_tightly | 41,424 | 870 | 2,883 |
| `Dilithium3` | | | |
| shake256_sw | 36,027 | 6,930 | 19,973 |
| shake256_lsu | 52,535 | 1,351 | 5,677 |
| shake256_tightly | 41,814 | 1,302 | 4,355 |
| `Dilithium5` | | | |
| shake256_sw | 35,983 | 11,486 | 33,064 |
| shake256_lsu | 52,203 | 2,069 | 8,640 |
| shake256_tightly | 41,504 | 2,088 | 6,932 |

Table 19. Power and Energy Consumption of CRYSTALS-Dilithium Keygen Using TurboSHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| `Dilithium2` | | | |
| t_shake256_sw | 35,637 | 2,564 | 7,309 |
| t_shake256_lsu | 50,764 | 885 | 3,594 |
| t_shake256_tightly | 41,442 | 869 | 2,881 |
| `Dilithium3` | | | |
| t_shake256_sw | 35,646 | 4,458 | 12,712 |
| t_shake256_lsu | 51,164 | 1,351 | 5,529 |
| t_shake256_tightly | 41,823 | 1,300 | 4,349 |
| `Dilithium5` | | | |
| t_shake256_sw | 35,552 | 7,330 | 20,847 |
| t_shake256_lsu | 50,905 | 2,069 | 8,425 |
| t_shake256_tightly | 41,436 | 2,085 | 6,911 |

Table 20. Power and Energy Consumption of CRYSTALS-Dilithium Sign Using SHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Dilithium2 | | | |
| shake256_sw | 35,934 | 12,915 | 37,127 |
| shake256_lsu | 52,284 | 6,880 | 28,777 |
| shake256_tightly | 41,551 | 6,741 | 22,407 |
| Dilithium3 | | | |
| shake256_sw | 35,823 | 20,544 | 58,875 |
| shake256_lsu | 52,384 | 11,318 | 47,430 |
| shake256_tightly | 41,776 | 11,175 | 37,347 |
| Dilithium5 | | | |
| shake256_sw | 36,091 | 26,315 | 75,978 |
| shake256_lsu | 52,302 | 13,091 | 54,774 |
| shake256_tightly | 41,832 | 12,984 | 43,451 |

Table 21. Power and Energy Consumption of CRYSTALS-Dilithium Sign Using TurboSHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Dilithium2 | | | |
| t_shake256_sw | 35,288 | 10,268 | 28,986 |
| t_shake256_lsu | 51,346 | 6,903 | 28,355 |
| t_shake256_tightly | 41,636 | 6,800 | 22,649 |
| Dilithium3 | | | |
| t_shake256_sw | 35,287 | 16,375 | 46,225 |
| t_shake256_lsu | 51,402 | 11,335 | 46,611 |
| t_shake256_tightly | 41,856 | 11,169 | 37,399 |
| Dilithium5 | | | |
| t_shake256_sw | 35,693 | 20,397 | 58,242 |
| t_shake256_lsu | 51,268 | 13,130 | 53,851 |
| t_shake256_tightly | 41,698 | 12,907 | 43,055 |

Table 22. Power and Energy Consumption of CRYSTALS-Dilithium Verify Using SHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Dilithium2 | | | |
| shake256_sw | 36,746 | 4,122 | 12,117 |
| shake256_lsu | 52,279 | 1,165 | 4,872 |
| shake256_tightly | 41,385 | 1,156 | 3,827 |
| Dilithium3 | | | |
| shake256_sw | 36,752 | 6,883 | 20,237 |
| shake256_lsu | 52,495 | 1,745 | 7,328 |
| shake256_tightly | 41,828 | 1,701 | 5,691 |
| Dilithium5 | | | |
| shake256_sw | 36,865 | 11,748 | 34,647 |
| shake256_lsu | 52,305 | 2,561 | 10,716 |
| shake256_tightly | 41,729 | 2,576 | 8,599 |

Table 23. Power and Energy Consumption of CRYSTALS-Dilithium Verify Using TurboSHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Dilithium2 | | | |
| t_shake256_sw | 36,149 | 2,820 | 8,155 |
| t_shake256_lsu | 51,066 | 1,165 | 4,759 |
| t_shake256_tightly | 41,425 | 1,155 | 3,827 |
| Dilithium3 | | | |
| t_shake256_sw | 36,152 | 4,594 | 13,286 |
| t_shake256_lsu | 51,203 | 1,745 | 7,147 |
| t_shake256_tightly | 41,739 | 1,699 | 5,673 |
| Dilithium5 | | | |
| t_shake256_sw | 36,240 | 7,671 | 22,239 |
| t_shake256_lsu | 51,036 | 2,561 | 10,456 |
| t_shake256_tightly | 41,732 | 2,572 | 8,586 |

Table 24. Power and Energy Consumption of Falcon Keygen Using SHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Falcon512 | | | |
| shake256_sw | 34,088 | 286,426 | 781,095 |
| shake256_lsu | 50,791 | 213,886 | 869,078 |
| shake256_tightly | 41,795 | 212,996 | 712,173 |
| Falcon1024 | | | |
| shake256_sw | 34,378 | 750,812 | 2,064,913 |
| shake256_lsu | 50,884 | 630,258 | 2,565,603 |
| shake256_tightly | 41,958 | 627,305 | 2,105,637 |

Table 25. Power and Energy Consumption of Falcon Keygen Using TurboSHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Falcon512 | | | |
| t_shake256_sw | 33,756 | 252,868 | 682,864 |
| t_shake256_lsu | 49,053 | 214,166 | 840,438 |
| t_shake256_tightly | 41,792 | 213,244 | 712,951 |
| Falcon1024 | | | |
| t_shake256_sw | 33,911 | 698,012 | 1,893,622 |
| t_shake256_lsu | 49,115 | 632,931 | 2,486,912 |
| t_shake256_tightly | 41,867 | 629,925 | 2,109,845 |

Table 26. Power and Energy Consumption of Falcon Sign Using SHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Falcon512 | | | |
| shake256_sw | 33,871 | 70,381 | 190,709 |
| shake256_lsu | 51,233 | 70,122 | 287,404 |
| shake256_tightly | 42,330 | 70,301 | 238,067 |
| Falcon1024 | | | |
| shake256_sw | 33,903 | 153,913 | 417,448 |
| shake256_lsu | 51,279 | 153,419 | 629,373 |
| shake256_tightly | 42,258 | 153,812 | 519,982 |

Table 27. Power and Energy Consumption of Falcon Sign Using TurboSHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Falcon512 | | | |
| t_shake256_sw | 33,865 | 70,256 | 190,337 |
| t_shake256_lsu | 49,588 | 70,124 | 278,184 |
| t_shake256_tightly | 42,331 | 70,302 | 238,076 |
| Falcon1024 | | | |
| t_shake256_sw | 33,894 | 153,681 | 416,709 |
| t_shake256_lsu | 49,557 | 153,420 | 608,242 |
| t_shake256_tightly | 42,288 | 153,813 | 520,355 |

Table 28. Power and Energy Consumption of Falcon Verify Using SHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Falcon512 | | | |
| shake256_sw | 35,508 | 585 | 1,661 |
| shake256_lsu | 52,269 | 351 | 1,467 |
| shake256_tightly | 42,839 | 350 | 1,199 |
| Falcon1024 | | | |
| shake256_sw | 35,453 | 1,206 | 3,420 |
| shake256_lsu | 52,138 | 745 | 3,107 |
| shake256_tightly | 42,828 | 741 | 2,538 |

Table 29. Power and Energy Consumption of Falcon Verify Using TurboSHAKE256

| Architecture | Power [µW] | Cycles [$x10^3$] | Energy [µJ] |
|---|---|---|---|
| Falcon512 | | | |
| t_shake256_sw | 35,199 | 472 | 1,329 |
| t_shake256_lsu | 50,549 | 351 | 1,419 |
| t_shake256_tightly | 42,845 | 349 | 1,196 |
| Falcon1024 | | | |
| t_shake256_sw | 35,212 | 987 | 2,780 |
| t_shake256_lsu | 50,456 | 745 | 3,007 |
| t_shake256_tightly | 42,996 | 741 | 2,548 |

Table 30. Power and Energy Consumption of CRYSTALS-Kyber Keygen Using SHAKE256

| Architecture | Power [µW] | Cycles | Energy [µJ] |
|---|---|---|---|
| Kyber512 | | | |
| shake256_sw | 36,124 | 1,144,653 | 3,307 |
| shake256_lsu | 53,124 | 327,542 | 1,392 |
| shake256_tightly | 42,072 | 328,496 | 1,105 |
| Kyber768 | | | |
| shake256_sw | 36,072 | 1,855,586 | 5,354 |
| shake256_lsu | 52,984 | 550,632 | 2,333 |
| shake256_tightly | 41,879 | 552,291 | 1,850 |
| Kyber1024 | | | |
| shake256_sw | 36,191 | 2,918,755 | 8,450 |
| shake256_lsu | 52,969 | 826,400 | 3,501 |
| shake256_tightly | 41,846 | 828,801 | 2,774 |

Table 31. Power and Energy Consumption of CRYSTALS-Kyber Keygen Using TurboSHAKE256

| Architecture | Power [µW] | Cycles | Energy [µJ] |
|---|---|---|---|
| Kyber512 | | | |
| t_shake256_sw | 35,801 | 789,173 | 2,260 |
| t_shake256_lsu | 51,894 | 327,481 | 1,359 |
| t_shake256_tightly | 42,085 | 328,171 | 1,104 |
| Kyber768 | | | |
| t_shake256_sw | 35,786 | 1,289,208 | 3,690 |
| t_shake256_lsu | 51,760 | 550,544 | 2,279 |
| t_shake256_tightly | 41,907 | 551,774 | 1,849 |
| Kyber1024 | | | |
| t_shake256_sw | 35,875 | 2,009,695 | 5,767 |
| t_shake256_lsu | 51,809 | 826,288 | 3,424 |
| t_shake256_tightly | 41,852 | 827,970 | 2,772 |

Table 32. Power and Energy Consumption of CRYSTALS-Kyber Encaps Using SHAKE256

| Architecture | Power [µW] | Cycles | Energy [µJ] |
|---|---|---|---|
| Kyber512 | | | |
| shake256_sw | 35,997 | 1,489,763 | 4,290 |
| shake256_lsu | 52,617 | 457,337 | 1,925 |
| shake256_tightly | 41,496 | 458,907 | 1,523 |
| Kyber768 | | | |
| shake256_sw | 35,955 | 2,377,552 | 6,838 |
| shake256_lsu | 52,674 | 713,484 | 3,006 |
| shake256_tightly | 41,573 | 715,715 | 2,380 |
| Kyber1024 | | | |
| shake256_sw | 36,044 | 3,560,411 | 10,266 |
| shake256_lsu | 52,681 | 1,025,291 | 4,321 |
| shake256_tightly | 41,653 | 1,028,323 | 3,426 |

Table 33. Power and Energy Consumption of CRYSTALS-Kyber Encaps Using TurboSHAKE256

| Architecture | Power [µW] | Cycles | Energy [µJ] |
|---|---|---|---|
| Kyber512 | | | |
| t_shake256_sw | 35,609 | 1,042,226 | 2,969 |
| t_shake256_lsu | 51,445 | 457,237 | 1,881 |
| t_shake256_tightly | 41,574 | 458,498 | 1,524 |
| Kyber768 | | | |
| t_shake256_sw | 35,658 | 1,653,361 | 4,716 |
| t_shake256_lsu | 51,459 | 713,341 | 2,936 |
| t_shake256_tightly | 41,540 | 715,054 | 2,376 |
| Kyber1024 | | | |
| t_shake256_sw | 35,722 | 2,454,087 | 7,013 |
| t_shake256_lsu | 51,442 | 1,025,137 | 4,218 |
| t_shake256_tightly | 41,619 | 1,027,312 | 3,420 |

Table 34. Power and Energy Consumption of CRYSTALS-Kyber Decaps Using SHAKE256

| Architecture | Power [µW] | Cycles | Energy [µJ] |
|---|---|---|---|
| Kyber512 | | | |
| shake256_sw | 35,700 | 1,465,204 | 4,184 |
| shake256_lsu | 52,526 | 642,028 | 2,697 |
| shake256_tightly | 41,459 | 643,292 | 2,133 |
| Kyber768 | | | |
| shake256_sw | 35,796 | 2,323,909 | 6,654 |
| shake256_lsu | 52,691 | 956,781 | 4,033 |
| shake256_tightly | 41,654 | 958,688 | 3,194 |
| Kyber1024 | | | |
| shake256_sw | 35,859 | 3,478,511 | 9,978 |
| shake256_lsu | 52,769 | 1,327,952 | 5,605 |
| shake256_tightly | 41,562 | 1,330,613 | 4,424 |

Table 35. Power and Energy Consumption of CRYSTALS-Kyber Decaps Using TurboSHAKE256

| Architecture | Power [µW] | Cycles | Energy [µJ] |
|---|---|---|---|
| Kyber512 | | | |
| t_shake256_sw | 35,390 | 1,109,724 | 3,141 |
| t_shake256_lsu | 51,420 | 641,948 | 2,640 |
| t_shake256_tightly | 41,543 | 642,967 | 2,136 |
| Kyber768 | | | |
| t_shake256_sw | 35,448 | 1,731,228 | 4,909 |
| t_shake256_lsu | 51,485 | 956,660 | 3,940 |
| t_shake256_tightly | 41,668 | 958,147 | 3,193 |
| Kyber1024 | | | |
| t_shake256_sw | 35,574 | 2,543,150 | 7,237 |
| t_shake256_lsu | 51,520 | 1,327,820 | 5,472 |
| t_shake256_tightly | 41,632 | 1,329,758 | 4,428 |

## References


[1] Aikata Aikata, Ahmet Can Mert, David Jacquemin, Amitabh Das, Donald Matthews, Santosh Ghosh, and Sujoy Sinha Roy. 2023. A unified cryptoprocessor for lattice-based signature and key-exchange. *IEEE Trans. Comput.* 72, 6 (2023), 1568–1580. https://doi.org/10.1109/TC.2022.3215064

[2] Dorian Amiet, Lukas Leuenberger, Andreas Curiger, and Paul Zbinden. 2020. FPGA-based SPHINCS$^+$ implementations: Mind the glitch. In *23rd Euromicro Conference on Digital System Design (DSD '20)*. IEEE, 229–237. https://doi.org/10.1109/DSD51259.2020.00046

[3] Jean-Philippe Aumasson, Daniel J. Bernstein, Ward Beullens, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Andreas Hülsing, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, and Bas Westerbaan. 2022. SPHINCS+—Submission to the 3rd round of the NIST Post-Quantum Project. v3.1. Online. https://sphincs.org/data/sphincs+-r3.1-specification.pdf

[4] Utsav Banerjee, Tenzin S. Ukyab, and Anantha P. Chandrakasan. 2019. Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols. *IACR Trans Cryptogr. Hardw. Embed. Syst.* 2019 (Aug. 2019), 17–61. https://doi.org/10.46586/tches.v2019.i4.17-61

[5] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. 2019. The SPHINCS+ signature framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. ACM. https://doi.org/10.1145/3319535.3363229

[6] Quentin Berthet, Andres Upegui, Laurent Gantel, Alexandre Duc, and Giulia Traverso. 2021. An area-efficient SPHINCS$^+$ post-quantum signature coprocessor. In *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPS Workshops '21)*. IEEE, 180–187. https://doi.org/10.1109/IPDPSW52791.2021.00034

[7] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. 2023. TurboSHAKE. IACR Cryptology ePrint Archive (2023), 342. https://eprint.iacr.org/2023/342.

[8] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. CRYSTALS-kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P '18)*. IEEE, 353–367.

[9] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. 2021. Ascon v1.2. https://ascon.iaik.tugraz.at/specification.html

[10] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018 (2018), 238–268. 10.13154/tches.v2018.i1.238-268

[11] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. 2018. Falcon: Fast-fourier lattice-based compact signatures over NTRU. Submission to the NIST's Post-Quantum Cryptography Standardization Process 36 (2018). https://falcon-sign.info/falcon.pdf

[12] Tim Fritzmann, Uzair Sharif, Daniel Müller-Gritschneder, Cezar Reinbrecht, Ulf Schlichtmann, and Johanna Sepulveda. 2019. Towards reliable and secure post-quantum co-processors based on RISC-V. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE '19)*. 1148–1153. https://doi.org/10.23919/DATE.2019.8715173

[13] Tim Fritzmann, Georg Sigl, and Johanna Sepúlveda. 2020. RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 4 (2020), 239–280. https://doi.org/10.13154/tches.v2020.i4.239-280

[14] Michael Gautschi, Pasquale Davide Schiavone, Andreas Traber, Igor Loi, Antonio Pullini, Davide Rossi, Eric Flamand, Frank K. Gürkaynak, and Luca Benini. 2017. Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices. *IEEE Trans. Very Large Scale Integr. Syst.* 25, 10 (2017), 2700–2713. https://doi.org/10.1109/TVLSI.2017.2654506

[15] Andreas Hülsing. 2013. W-OTS+—Shorter signatures for hash-based signature schemes. In *Progress in Cryptology (AFRICACRYPT '13)*, Amr Youssef, Abderrahmane Nitaj, and Aboul Ella Hassanien (Eds.). Springer, Berlin, 173–188.

[16] Matthias J. Kannwischer, Markus Krausz, Richard Petri, and Shang-Yi Yang. 2024. pqm4: Benchmarking NIST Additional Post-Quantum Signature Schemes on Microcontrollers. Cryptology ePrint Archive, Paper 2024/112. https://eprint.iacr.org/2024/112

[17] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. [n. d.]. PQM4: Post-quantum crypto library for the ARM Cortex-M4. https://github.com/mupq/pqm4, as of commit 918f379.

[18] Patrick Karl, Jonas Schupp, Tim Fritzmann, and Georg Sigl. 2023. Post-quantum signatures on RISC-V with hardware acceleration. *ACM Trans. Embed. Comput. Syst.* 23, 2 (2023), 1–23. https://doi.org/10.1145/3579092

[19] Patrick Karl, Jonas Schupp, and Georg Sigl. 2024. The impact of hash primitives and communication overhead for hardware-accelerated SPHINCS+. In *Proceedings of the 15th International Workshop on Constructive Side-channel Analysis and Secure Design (COSADE '24) (Lecture Notes in Computer Science, Vol. 14595)*, Romain Wacquez and Naofumi Homma (Eds.). Springer, 221–239. https://doi.org/10.1007/978-3-031-57543-3_12

[20] Dustin Moody. 2022. NIST PQC: Looking into the Future. https://csrc.nist.gov/Presentations/2022/nist-pqc-looking-into-the-future

[21] National Institute of Standards and Technology. 2023. *Module-lattice-based Digital Signature Standard*. https://doi.org/10.6028/nist.fips.204.ipd

[22] National Institute of Standards and Technology. 2023. *Module-lattice-based Key-encapsulation Mechanism Standard*. https://doi.org/10.6028/nist.fips.203.ipd

[23] National Institute of Standards and Technology. 2023. *Stateless Hash-based Digital Signature Standard*. https://doi.org/10.6028/nist.fips.205.ipd

[24] National Institute of Standards and Technology. 2022. Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process. https://csrc.nist.gov/Projects/pqc-dig-sig/standardization

[25] National Institute of Standards and Technology. 2022. Status Report on the Third Round of the NIST Post-Quantum Cryptography Stadardization Process. https://doi.org/10.6028/NIST.IR.8413-upd1

[26] National Institute of Standards and Technology. 2015. *SHA-3 Standard: Permutation-based Hash and Extendable-output Functions*. Technical Report. https://doi.org/10.6028/nist.fips.202

[27] Markku-Juhani O. Saarinen. 2024. Accelerating SLH-DSA by two orders of magnitude with a single hash unit. In *Proceedings of the 44th Annual International Cryptology Conference on Advances in Cryptology (CRYPTO '24)* , . To appear. https://eprint.iacr.org/2024/367. Available as IACR ePrint Report 2024/367.

[28] Stefan Steinegger and Robert Primas. 2020. A fast and compact RISC-V accelerator for ascon and friends. In *19th International Conference on Smart Card Research and Advanced Applications (CARDIS '20), Virtual Event, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 12609)*, Pierre-Yvan Liardet and Nele Mentens (Eds.). Springer, 53–67. https://doi.org/10.1007/978-3-030-68487-7_4

[29] Meltem Sonmez Turan. 2023. *Status Report on the Final Round of the NIST Lightweight Cryptography Standardization Process*. https://doi.org/10.6028/nist.ir.8454

[30] Alexander Wagner, Felix Oberhansl, and Marc Schink. 2022. To be, or not to be stateful: Post-quantum secure boot using hash-based signatures. In *Proceedings of the 2022 Workshop on Attacks and Solutions in Hardware Security (ASHES '22)*, Chip-Hong Chang, Ulrich Rührmair, Debdeep Mukhopadhyay, and Domenic Forte (Eds.). ACM, 85–94. https://doi.org/10.1145/3560834.3563831

[31] Wen Wang, Shanquan Tian, Bernhard Jungk, Nina Bindel, Patrick Longa, and Jakub Szefer. 2020. Parameterized hardware accelerators for lattice-based cryptography and their application to the HW/SW co-design of qTESLA. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2020, 3 (Jun. 2020), 269–306. https://doi.org/10.13154/tches.v2020.i3.269-306

Received 3 March 2024; revised 4 June 2024; accepted 18 June 2024